

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY  
UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



## **BIG DATA (CO3137)**

---

### **Movie Recommendation System**

Applying Big Data and MLOps

### **Assignment Report**

---

Advisor: Nguyễn Quang Hùng  
Students: Phan Thanh Tấn - 22113076.

HO CHI MINH CITY, FEBRUARY 2026

## Tóm tắt

Báo cáo tóm tắt kiến trúc, pipeline dữ liệu, thực nghiệm và triển khai hệ thống gợi ý phim trên nền tảng cloud-native.

Báo cáo trình bày quá trình thiết kế và đánh giá một hệ thống gợi ý phim ứng dụng Dữ liệu lớn và MLOps, tập trung vào khả năng triển khai thực tế với chi phí thấp nhưng vẫn duy trì độ trễ phản hồi phù hợp cho trải nghiệm người dùng. Hệ thống kết hợp Cloudflare R2 cho lưu trữ dữ liệu, Hugging Face Spaces cho lớp ứng dụng, LanceDB cho tìm kiếm vector, và Groq API cho tác tử hội thoại dựa trên LLM.

Về mặt thuật toán, đề án khai thác những ngữ nghĩa, cá nhân hóa giả lập theo hồ sơ người dùng, lớp tinh chỉnh xếp hạng, và kiến trúc Retrieval-Augmented Generation để tạo ra các đề xuất có khả năng giải thích. Phần đánh giá sử dụng các chỉ số Recall@10, NDCG@10 và độ trễ truy vấn; dashboard hiện tại ghi nhận Recall@10 xấp xỉ 0.86, NDCG@10 xấp xỉ 0.79, với độ trễ truy vấn LanceDB khoảng 45 ms.

**Mã nguồn:** [github.com/thanhtan2210/Big-Data-MLOps-System](https://github.com/thanhtan2210/Big-Data-MLOps-System) [20].

# TABLE OF CONTENTS

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Tóm tắt</b>                                                                 | <b>2</b>  |
| <b>1 Tổng quan đề tài</b>                                                      | <b>5</b>  |
| 1.1 Bối cảnh và động lực nghiên cứu . . . . .                                  | 5         |
| 1.2 Lý do lựa chọn đề tài . . . . .                                            | 7         |
| 1.3 Mục tiêu đề tài . . . . .                                                  | 8         |
| 1.4 Phạm vi và giới hạn đề tài . . . . .                                       | 9         |
| 1.5 Cấu trúc báo cáo . . . . .                                                 | 11        |
| <b>2 Các công trình liên quan và nền tảng lý thuyết</b>                        | <b>12</b> |
| 2.1 Hệ thống gợi ý (Recommendation Systems) . . . . .                          | 12        |
| 2.1.1 Collaborative Filtering (Lọc Cộng tác) . . . . .                         | 12        |
| 2.1.2 Content-Based Filtering (Lọc dựa trên Nội dung) . . . . .                | 13        |
| 2.1.3 Hybrid Approach và Neural Collaborative Filtering . . . . .              | 14        |
| 2.2 Semantic Search và Vector Embeddings . . . . .                             | 14        |
| 2.2.1 Transformer-based Embeddings . . . . .                                   | 15        |
| 2.2.2 Vector Database và Approximate Nearest Neighbor (ANN) . . . . .          | 15        |
| 2.3 Large Language Models và RAG . . . . .                                     | 16        |
| 2.3.1 Function Calling trong LLM . . . . .                                     | 16        |
| 2.3.2 Retrieval-Augmented Generation (RAG) . . . . .                           | 17        |
| 2.4 MLOps và Serverless Deployment . . . . .                                   | 18        |
| <b>3 Phát biểu bài toán và các thách thức</b>                                  | <b>19</b> |
| 3.1 Phát biểu bài toán chính thức . . . . .                                    | 19        |
| 3.1.1 Định nghĩa bài toán . . . . .                                            | 19        |
| 3.1.2 Các sub-task cần giải quyết . . . . .                                    | 20        |
| 3.2 Thách thức kỹ thuật . . . . .                                              | 20        |
| 3.2.1 Thách thức 1: Quy mô dữ liệu vs. RAM giới hạn . . . . .                  | 20        |
| 3.2.2 Thách thức 2: Bất ổn của DataFusion SQL Parser . . . . .                 | 21        |
| 3.2.3 Thách thức 3: Xung đột phiên bản thư viện . . . . .                      | 22        |
| 3.2.4 Thách thức 4: Ngăn chặn Hallucination của LLM . . . . .                  | 22        |
| 3.2.5 Thách thức 5: Giới hạn Quota API và High Availability . . . . .          | 23        |
| 3.3 Thách thức phi kỹ thuật . . . . .                                          | 23        |
| 3.3.1 Chi phí hạ tầng và Chuyển dịch mô hình triển khai . . . . .              | 23        |
| 3.3.2 Bài toán Khởi động Nguội (Cold Start Problem) . . . . .                  | 24        |
| <b>4 Kiến trúc Hệ thống và Triển khai Kỹ thuật Hiện tại</b>                    | <b>24</b> |
| 4.1 Tổng quan kiến trúc . . . . .                                              | 24        |
| 4.2 Đường ống Tiền xử lý Dữ liệu (Data Pipeline) . . . . .                     | 25        |
| 4.2.1 Dòng dữ liệu và thống kê sau từng bước trong notebook . . . . .          | 25        |
| 4.2.2 Tải dữ liệu và tinh lọc bản ghi . . . . .                                | 26        |
| 4.2.3 Kỹ thuật Tăng cường Văn bản Nhúng (Text Embedding Enhancement) . . . . . | 26        |
| 4.2.4 Xây dựng Không gian LanceDB và Đánh giá MLOps . . . . .                  | 27        |
| 4.2.5 Schema LanceDB và tham số chỉ mục . . . . .                              | 27        |
| 4.2.6 Tính tái lập và quản lý bí mật thực nghiệm . . . . .                     | 28        |
| 4.3 Động cơ Tìm kiếm Vector (Vector Search Engine) . . . . .                   | 28        |
| 4.3.1 Khởi tạo và Tối ưu Lọc Siêu dữ liệu (Pandas Filtering) . . . . .         | 29        |
| 4.3.2 Cá nhân hóa Giả lập (Pseudo-Tower Personalization) . . . . .             | 29        |
| 4.3.3 Tinh chỉnh xếp hạng (Reranker Layer) . . . . .                           | 29        |



|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.4      | Tác tử Trí tuệ Nhân tạo (LLM Agent) . . . . .                       | 30        |
| 4.4.1    | Mô hình và Giao thức Gọi hàm (Function Calling) . . . . .           | 30        |
| 4.4.2    | Kiến trúc RAG nguyên bản (Retrieval-Augmented Generation) . . . . . | 30        |
| 4.4.3    | Bộ nhớ Thực thể và Chế độ Dự phòng thông minh . . . . .             | 31        |
| 4.5      | Giao diện Người dùng và Phân tích Trực quan (Streamlit) . . . . .   | 31        |
| 4.5.1    | Bảng điều khiển Phân tích (Analytics Dashboard) . . . . .           | 31        |
| 4.5.2    | Tối ưu hóa Trải nghiệm Người dùng (UX) . . . . .                    | 31        |
| 4.5.3    | Minh họa triển khai và trải nghiệm người dùng . . . . .             | 32        |
| <b>5</b> | <b>Thực nghiệm và đánh giá</b>                                      | <b>34</b> |
| 5.1      | Môi trường thực nghiệm . . . . .                                    | 34        |
| 5.2      | Bộ dữ liệu thực nghiệm . . . . .                                    | 34        |
| 5.3      | Phân tích khám phá dữ liệu từ notebook . . . . .                    | 35        |
| 5.4      | Chỉ số đánh giá . . . . .                                           | 36        |
| 5.4.1    | Chỉ số Recall@K . . . . .                                           | 38        |
| 5.4.2    | Chỉ số NDCG@K (Normalized Discounted Cumulative Gain) . . . . .     | 38        |
| 5.4.3    | Độ trễ và Thông lượng (Latency & Throughput) . . . . .              | 38        |
| 5.5      | Kết quả thực nghiệm . . . . .                                       | 38        |
| 5.6      | So sánh kiến trúc cũ và mới . . . . .                               | 39        |
| 5.7      | Phân tích lỗi và hạn chế . . . . .                                  | 40        |
| <b>6</b> | <b>Kết luận và Hướng phát triển</b>                                 | <b>42</b> |
| 6.1      | Tổng kết kết quả đạt được . . . . .                                 | 42        |
| 6.2      | Đóng góp của đề tài . . . . .                                       | 43        |
| 6.3      | Hạn chế còn tồn tại . . . . .                                       | 44        |
| 6.4      | Hướng phát triển tương lai . . . . .                                | 45        |
| 6.5      | Lời kết . . . . .                                                   | 46        |
|          | <b>Tài liệu tham khảo</b>                                           | <b>47</b> |

# 1 Tổng quan đề tài

Phần này trình bày một cách toàn diện và sâu sắc về bối cảnh hình thành, động lực nghiên cứu, cũng như những lý luận cốt lõi dẫn đến việc lựa chọn đề tài. Thông qua việc phân tích sự bùng nổ của dữ liệu lớn (Big Data) trong kỷ nguyên số và nhu cầu cấp thiết về các hệ thống gợi ý (Recommender Systems) thông minh, phần này làm nổi bật giá trị học thuật và tính ứng dụng thực tiễn của dự án. Tiếp đó, các mục tiêu cụ thể, phạm vi nghiên cứu, và các giới hạn kỹ thuật sẽ được định hình rõ ràng nhằm tạo tiền đề vững chắc cho việc thiết kế và phát triển kiến trúc hệ thống ở các phần sau. Cuối cùng, cấu trúc tổng thể của toàn bộ báo cáo sẽ được tóm tắt, giúp người đọc dễ dàng theo dõi và nắm bắt mạch logic của nghiên cứu.

## 1.1 Bối cảnh và động lực nghiên cứu

Trong kỷ nguyên kỹ thuật số hiện đại, sự phát triển bùng nổ của hạ tầng viễn thông, mạng Internet vạn vật (IoT), và các thiết bị di động thông minh đã mở ra một chương mới cho ngành công nghiệp giải trí trực tuyến. Sự chuyển dịch mạnh mẽ từ các phương thức phát sóng truyền thống (truyền hình cáp, đĩa vật lý) sang các nền tảng phân phối nội dung số theo yêu cầu (Video on Demand - VoD) và truyền phát trực tuyến (Streaming) đã tái định hình hoàn toàn thói quen tiêu thụ thông tin của người dùng. Hàng loạt các tập đoàn công nghệ lớn như Netflix, YouTube, Amazon Prime Video, Disney+, hay Spotify không chỉ đóng vai trò là nhà cung cấp nội dung, mà còn là những hệ thống thu thập dữ liệu quy mô lớn. Mỗi giây trôi qua, hàng tỷ hành vi tương tác của người dùng—from việc nhấp chuột, xem lướt qua, dừng lại, tua nhanh, cho đến việc để lại các bình luận và điểm số đánh giá—đều được hệ thống ghi nhận, lưu trữ và phân tích. Sự gia tăng theo cấp số nhân về cả khối lượng (Volume), tốc độ (Velocity) và tính đa dạng (Variety) của các luồng sự kiện này đã tạo ra một nguồn tri thức lớn, thường được biết đến dưới khái niệm Dữ liệu lớn (Big Data). Nguồn dữ liệu này chứa các tín hiệu ẩn (latent signals) phản ánh chi tiết và phức tạp về sở thích, xu hướng tâm lý, và thói quen tiêu dùng của từng cá nhân trong xã hội.

Tuy nhiên, sự bùng nổ của khối lượng nội dung số cũng đồng thời kéo theo một hệ lụy đáng kể về mặt tâm lý và trải nghiệm người dùng, đó chính là hội chứng "Quá tải thông tin" (Information Overload). Khi đứng trước một thư viện chứa hàng triệu bộ phim, hàng chục ngàn chương trình truyền hình và vô số các video ca nhạc, người dùng dễ rơi vào trạng thái mệt mỏi nhận thức (cognitive overload) khi phải đưa ra quyết định. Nghịch lý của sự lựa chọn (Paradox of Choice) chỉ ra rằng, càng có nhiều lựa chọn, người dùng càng mất nhiều thời gian để tìm kiếm, và cuối cùng thường dẫn đến cảm giác chán nản, không hài lòng, thậm chí từ bỏ việc sử dụng dịch vụ. Các công cụ tìm kiếm truyền thống (Search Engines) dựa trên việc khớp từ khóa (keyword matching) khó xử lý hiệu quả vấn đề này, bởi vì chúng đòi hỏi người dùng phải biết chính xác mình đang muốn tìm kiếm thứ gì. Trong bối cảnh đó, các Hệ thống Gợi ý (Recommender Systems) ra đời và nhanh chóng trở thành một công cụ lọc chủ động (active filter) không thể thiếu. Thay vì đợi người dùng đặt câu hỏi, hệ thống gợi ý phân tích bối cảnh và lịch sử tương tác để tiên đoán và tự động đề xuất những nội dung có xác suất được yêu thích cao nhất. Một hệ thống gợi ý hiệu quả không chỉ giúp tiết kiệm thời gian, cá nhân hóa trải nghiệm người dùng ở mức độ vi mô, mà còn đóng vai trò quan trọng trong việc tối ưu hóa tỷ lệ giữ chân khách hàng (Retention Rate), tăng cường mức độ gắn kết (Engagement), và mang lại hàng tỷ đô la doanh thu cho các tập đoàn công nghệ toàn cầu.

Để hiện thực hóa khả năng cá nhân hóa trải nghiệm ở quy mô rất lớn, sự kết hợp giữa Dữ liệu lớn (Big Data) và Học máy (Machine Learning) là điều kiện tiên quyết. Nếu như Big Data cung cấp nguồn nguyên liệu thô dồi dào, thì Machine Learning chính là động cơ tinh chế,

trích xuất ra các quy luật, khuôn mẫu ẩn (hidden patterns) phức tạp mà các hệ thống luật (rule-based systems) tĩnh không thể nào nắm bắt được. Quá trình phát triển của các thuật toán gợi ý đã trải qua nhiều giai đoạn, từ những phương pháp thô sơ như Lọc cộng tác dựa trên bộ nhớ (Memory-based Collaborative Filtering), Lọc dựa trên nội dung (Content-based Filtering), cho đến các kỹ thuật Phân rã ma trận (Matrix Factorization). Gần đây nhất, sự trỗi dậy của Học sâu (Deep Learning) và các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) đã nâng tầm khả năng biểu diễn ngữ nghĩa (Semantic Representation) của cả người dùng và sản phẩm vào trong các Không gian Vector đa chiều (Vector Spaces). Việc nhúng (embedding) các metadata phức tạp thành vector cho phép máy tính hiểu được sắc thái ngữ cảnh, đồng thời so sánh độ tương đồng giữa hàng triệu thực thể chỉ trong vài phần nghìn giây. Điều này mở ra khả năng xây dựng các hệ thống gợi ý lai (Hybrid Recommender Systems), nơi mà trí tuệ nhân tạo có thể giải quyết trọn vẹn cả bài toán "khởi động nguội" (cold-start problem) của các bộ phim mới, lẫn tính thưa thớt (sparsity) của ma trận dữ liệu tương tác.

Trong khuôn khổ của các nghiên cứu học thuật và đánh giá hiệu năng thuật toán gợi ý, bộ dữ liệu MovieLens 25M do tổ chức GroupLens Research (Đại học Minnesota) thu thập và công bố từ lâu đã được công nhận là một tiêu chuẩn vàng (gold standard benchmark). Bộ dữ liệu này chứa đựng hơn 25 triệu lượt đánh giá (ratings) từ khoảng 162.000 người dùng đối với hơn 62.000 bộ phim khác nhau, thu thập liên tục trong khoảng thời gian từ năm 1995 đến năm 2019 [6, 5]. Vượt xa khỏi khuôn khổ của một bảng đánh giá 5 sao thông thường, MovieLens 25M còn cung cấp một khối lượng metadata quy mô lớn bao gồm hơn 1 triệu nhãn dán tự do (user-generated tags) và hệ thống điểm số gen (genome-scores) phản ánh mức độ liên quan của 1.128 thể từ vựng (tags) đối với từng bộ phim. Khối lượng và sự đa dạng của tập dữ liệu này tạo ra một bài toán thử thách thực sự cho bất kỳ kỹ sư dữ liệu nào. Nó đòi hỏi việc thiết kế các kỹ thuật xử lý dữ liệu lớn tinh vi để dọn dẹp, chuẩn hóa, và chuyển đổi thông tin thành các ma trận vector có thể tính toán được, đồng thời duy trì được tính toàn vẹn của các ngữ nghĩa phức tạp ẩn sâu bên trong các nhãn dán. Chính sự phong phú này khiến MovieLens 25M trở thành môi trường lý tưởng nhất để kiểm chứng hiệu quả của các kiến trúc hệ thống tìm kiếm ngữ nghĩa hiện đại.

Mặc dù các thuật toán học máy liên tục đạt được những kỷ lục mới về độ chính xác trên môi trường thử nghiệm, ngành công nghiệp công nghệ thông tin đang chứng kiến một sự dịch chuyển mạnh mẽ từ việc chỉ tập trung vào mô hình học máy (Model-centric) sang việc tập trung vào toàn bộ vòng đời vận hành hệ thống (System-centric), hay còn gọi là MLOps (Machine Learning Operations). Các nghiên cứu chỉ ra rằng, mã nguồn của mô hình học máy chỉ chiếm một phần rất nhỏ trong toàn bộ cơ sở mã (codebase) của một hệ thống AI thực tế; phần còn lại bao gồm các cấu trúc hạ tầng dành cho việc thu thập dữ liệu, phân tích đặc trưng, quản lý tài nguyên, phục vụ mô hình (model serving), và giám sát hiệu năng liên tục [14]. Đặc biệt, chi phí duy trì các máy chủ vật lý, cơ sở dữ liệu quan hệ, và các cụm xử lý GPU quy mô lớn 24/7 đang trở thành gánh nặng tài chính rất lớn, ngay cả đối với các doanh nghiệp lớn. Do đó, xu hướng MLOps hiện đại đang tiến mạnh về phía các kiến trúc phi máy chủ (Serverless) và điện toán đám mây nguyên bản (Cloud-native). Việc xây dựng các hệ thống có khả năng tự động mở rộng theo nhu cầu (auto-scaling) hoặc tạm ngưng hoàn toàn để đưa chi phí vận hành về 0 (scale-to-zero) khi không có truy cập, đang trở thành tiêu chuẩn mới. Động lực nghiên cứu của đề tài này chính là việc nắm bắt xu hướng đó: không chỉ giải bài toán lý thuyết trên dữ liệu lớn, mà còn phải hiện thực hóa nó thành một sản phẩm có thể phục vụ người dùng thực tế với hiệu suất cao và chi phí tối thiểu.

Thông qua việc đánh giá toàn diện các yếu tố từ sự bùng nổ dữ liệu, bài toán quá tải thông tin, tiềm năng của trí tuệ nhân tạo, cho đến những thách thức trong việc triển khai MLOps,

bức tranh tổng thể về bối cảnh nghiên cứu đã được định hình rõ nét, tạo cơ sở logic vững chắc để tiến tới việc xác định lý do lựa chọn và mục tiêu cụ thể của đề tài.

## 1.2 Lý do lựa chọn đề tài

Một trong những khoảng trống lớn nhất và rõ rệt nhất trong lĩnh vực nghiên cứu và giáo dục trí tuệ nhân tạo hiện nay là sự chênh lệch (gap) giữa các mô hình học thuật mang tính thử nghiệm (experimental models) và các hệ thống phần mềm hoàn chỉnh có khả năng chịu tải trong môi trường sản xuất (production-ready systems). Phần lớn các đồ án và nghiên cứu học máy thường chỉ dừng lại ở việc thiết kế thuật toán, chạy thử nghiệm trên các tệp tin tĩnh (static CSV files) bên trong môi trường Jupyter Notebook hoặc Google Colab cục bộ. Khi kết thúc quá trình huấn luyện và thu được các chỉ số độ chính xác (như RMSE, MAE, hay Recall), dự án thường được coi là hoàn tất. Tuy nhiên, trong môi trường công nghiệp thực tế, một mô hình dự đoán chính xác sẽ hoàn toàn vô giá trị nếu nó không được tích hợp vào một đường ống dữ liệu (Data Pipeline) liền mạch, không có khả năng nhận dữ liệu thô liên tục, không có cơ chế chuyển đổi đặc trưng thời gian thực, và không có một giao diện lập trình ứng dụng (API) hoặc giao diện người dùng (UI) tối ưu để trả về kết quả trong thời gian tính bằng mili-giây. Chính khoảng cách giữa "phát triển mô hình" và "kỹ thuật phần mềm" này là lý do cốt lõi thúc đẩy việc lựa chọn đề tài. Đề tài mong muốn xây dựng một hệ thống từ đầu đến cuối (End-to-End System), kết nối hoàn chỉnh vòng đời từ dữ liệu thô (Raw Data) đến trải nghiệm người dùng cuối (End-user Experience), chứng minh rằng việc triển khai AI là một quy trình kỹ thuật hệ thống phức tạp chứ không chỉ là toán học tối ưu.

Sự kết hợp giữa Dữ liệu lớn (Big Data), Tìm kiếm Không gian Vector (Vector Search), và Tác tử Mô hình Ngôn ngữ Lớn (LLM Agent) là một hướng tiếp cận tiêu biểu trong các hệ thống trí tuệ nhân tạo hiện đại. Việc lựa chọn kết hợp ba công nghệ này trong cùng một đề tài không phải là một sự lắp ghép ngẫu nhiên, mà nhằm bổ sung cho nhau về chức năng. Dữ liệu lớn cung cấp nền tảng sự thật (Ground Truth) có giá trị thông qua hàng chục triệu bản ghi tương tác của MovieLens, đảm bảo tính đại diện và khách quan của hành vi con người. Công nghệ Vector Search (đại diện bởi LanceDB và SentenceTransformer) giải quyết bài toán truy xuất thông tin (Retrieval) với tốc độ nhanh và độ chính xác ngữ nghĩa cao, biến các chuỗi văn bản phức tạp thành các mảng số học có thể đo lường khoảng cách [9, 16]. Tuy nhiên, cả Big Data và Vector Search đều chủ yếu trả về kết quả truy xuất, chưa hỗ trợ tốt khả năng diễn giải và giao tiếp tự nhiên. Vì vậy, LLM Agent (đại diện bởi mô hình Llama-3.3-70b) được bổ sung như một lớp giao tiếp và diễn giải, tiếp nhận kết quả từ Vector Database, sử dụng khả năng hiểu ngôn ngữ (Reasoning) để giải thích cho người dùng lý do tại sao bộ phim đó lại được đề xuất, tạo ra một trải nghiệm hội thoại (Conversational Experience) cá nhân hóa hơn. Đề tài lựa chọn kiến trúc này để chứng minh rằng, trí tuệ nhân tạo hiện đại phải là sự giao thoa ổn định giữa năng lực xử lý số liệu tĩnh và khả năng sáng tạo ngôn ngữ động.

Một lý do mang tính chiến lược khác dẫn đến việc hình thành đề tài là sự chuyển dịch kiến trúc hạ tầng từ mô hình máy chủ vật lý (On-premise / IaaS) tốn kém sang mô hình **Serverless Cloud-native**. Các hệ thống MLOps và Big Data truyền thống thường yêu cầu duy trì liên tục các cụm máy chủ (Clusters) cấu hình cao, chạy nền tảng Apache Kafka, cơ sở dữ liệu PostgreSQL, các kho lưu trữ Object Storage riêng biệt (như MinIO), và các khung phục vụ mô hình (Model Serving Frameworks) nặng nề như BentoML. Việc để các cụm máy chủ này chạy liên tục (Always-on) 24/7 tiêu tốn một lượng ngân sách rất lớn, hoàn toàn nằm ngoài tầm với của sinh viên, nhà nghiên cứu độc lập, hay các doanh nghiệp khởi nghiệp (Startups). Đề tài này mạnh dạn từ bỏ lối mòn đó, lựa chọn một hướng đi đột phá: kiến trúc hoàn toàn không có máy chủ trạng thái (Stateless Serverless). Bằng việc lưu trữ toàn bộ dữ liệu trên Cloudflare



R2 (nền tảng miễn phí bằng thông tải ra), triển khai mã nguồn trên Hugging Face Spaces (nền tảng tự động ngắt kết nối khi không sử dụng), và sử dụng cơ sở dữ liệu vector nhúng LanceDB (hoạt động trực tiếp trên RAM cục bộ), hệ thống đã giải quyết thành công bài toán chi phí [18, 17, 16].

Ý nghĩa thực tiễn của đề tài nằm ở việc nó tạo ra một **Khuôn mẫu Kiến trúc (Architectural Blueprint)** có khả năng tái sử dụng cao. Bất kỳ sinh viên, nhà nghiên cứu, hay kỹ sư phần mềm nào cũng có thể tham khảo kiến trúc này để tự mình triển khai một hệ thống trí tuệ nhân tạo cấp độ doanh nghiệp (Enterprise-grade) phục vụ cho các bài toán phân tích và gợi ý phức tạp mà không cần phải đầu tư hàng ngàn đô la vào hạ tầng phần cứng hay dịch vụ đám mây đắt đỏ. Đây là một nỗ lực có ý nghĩa nhằm "bình dân hóa" (Democratize) công nghệ Big Data và MLOps, mang hiệu quả của AI hiện đại đến gần hơn với cộng đồng mã nguồn mở và những người đam mê công nghệ với mức ngân sách tiệm cận bằng 0 (Zero-cost infrastructure). Sự kết hợp giữa tính học thuật trong xử lý dữ liệu và tính ứng dụng thực tế trong tối ưu hóa kiến trúc là định hướng xuyên suốt toàn bộ đề tài này.

Những lý do được phân tích ở trên không chỉ giải thích bối cảnh xuất phát điểm, mà còn định hình rõ ràng các đường hướng phát triển kỹ thuật. Trên cơ sở đó, các mục tiêu cụ thể và định lượng của dự án được thiết lập nhằm biến những lý tưởng này thành các thước đo có thể đánh giá và kiểm chứng được.

### 1.3 Mục tiêu đề tài

Mục tiêu cốt lõi của đề tài không chỉ dừng lại ở việc xây dựng một mô hình toán học dự đoán hành vi, mà là kiến tạo một hệ sinh thái phần mềm toàn diện, minh bạch, có khả năng tự động hóa cao và tối ưu hóa tối đa về mặt chi phí. Để đạt được sứ mệnh đó, đề tài đặt ra 5 mục tiêu cụ thể, được thiết kế theo tư duy phát triển phần mềm Agile và quy chuẩn MLOps hiện đại.

**1. Xây dựng đường ống dữ liệu toàn trình (End-to-End Data Pipeline) xử lý 25 triệu lượt đánh giá.** Mục tiêu đầu tiên là thiết lập một quy trình Trích xuất - Biến đổi - Tải (ETL - Extract, Transform, Load) tự động và bền bỉ, chạy trên nền tảng tính toán đám mây Google Colab. Đường ống này phải có khả năng tiếp nhận tập dữ liệu rất lớn MovieLens 25M từ các tệp tin lưu trữ thô, sau đó thực hiện quá trình làm sạch dữ liệu nghiêm ngặt. Cụ thể, hệ thống phải tự động lọc bỏ các bộ phim chất lượng thấp, có dưới 50 lượt tương tác, nhằm giảm nhiễu cho mô hình. Quan trọng hơn, hệ thống phải thực hiện các phép kết nối bảng (Table Joins) phức tạp bằng thư viện Pandas, hợp nhất thông tin từ *movies.csv*, *ratings.csv*, *tags.csv*, và *genome-scores.csv* để tạo ra các cấu trúc dữ liệu phẳng. Điểm nhấn của đường ống này là việc thiết kế thuật toán Tăng cường Ván bản Nhúng (Text Embedding Enhancement), nơi các metadata và điểm số chất lượng được biến đổi thành một chuỗi ngữ nghĩa làm giàu (Enriched Text), chuẩn bị nguồn nguyên liệu đầu vào tinh khiết nhất cho quá trình mã hóa vector.

**2. Thiết kế Cơ sở dữ liệu Vector với cấu trúc tĩnh cho tìm kiếm ngữ nghĩa tốc độ cao.** Mục tiêu thứ hai tập trung vào việc lưu trữ và truy xuất thông tin không gian đa chiều hiệu quả. Hệ thống cần tích hợp mô hình học sâu SentenceTransformer (all-MiniLM-L6-v2) để mã hóa các chuỗi văn bản làm giàu thành không gian vector 384 chiều tĩnh. Tiếp đó, cơ sở dữ liệu nhúng LanceDB được triển khai với một Lược đồ (Schema) khắt khe do pyarrow định nghĩa, ràng buộc cấu trúc mảng danh sách kích thước cố định FixedSizeList(384). Việc này nhằm loại trừ hoàn toàn các rủi ro về xung đột kích thước hình học (Tensor shape mismatch) khi giao tiếp với phần lõi lập trình bằng Rust của LanceDB. Dữ liệu vector sau khi được thiết lập chỉ mục lượng tử hóa mảng tích (IVF-PQ) hoặc tìm kiếm tuần tự (Flat Search) sẽ được nén thành các tệp tin tĩnh di động (Portable Artifacts), sẵn sàng để phân phối trên mạng lưới Internet với tốc độ độ trễ dưới 50 mili-giây cho mỗi truy vấn tìm kiếm Láng giềng gần nhất



(KNN).

**3. Phát triển Tác tử LLM (AI Agent) với khả năng Gọi hàm (Function Calling) và cơ chế RAG.** Mục tiêu thứ ba hướng tới việc xóa bỏ ranh giới giao tiếp thiếu linh hoạt giữa máy móc và con người. Hệ thống cần triển khai một Mô hình Ngôn ngữ Lớn (LLM) cấp độ chuyên gia, cụ thể là **Llama-3.3-70b-versatile** thông qua nền tảng suy luận tốc độ cao Groq API. LLM này không được phép hoạt động như một "mô hình sinh văn bản độc lập" sinh văn bản ảo giác (Hallucination), mà phải được kiến trúc thành một Tác tử thông minh (AI Agent). Nó phải được trang bị khả năng Gọi hàm (Function Calling) với 5 công cụ chuyên biệt, cho phép nó tự động suy luận để quyết định khi nào cần tra cứu cơ sở dữ liệu LanceDB. Hơn nữa, việc áp dụng mẫu thiết kế RAG (Retrieval-Augmented Generation) là bắt buộc. Toàn bộ thông tin trả về từ LLM phải được cung cấp căn cứ (Grounded) hoàn toàn vào các bối cảnh (Context) thực tế được truy xuất từ cơ sở dữ liệu vector. Đồng thời, Tác tử phải duy trì được một Bộ nhớ Thực thể (Entity Memory) trong suốt phiên làm việc (Session) để ghi nhớ sở thích cá nhân hóa của người dùng.

**4. Triển khai toàn bộ hệ thống lên môi trường đám mây (Cloud-native Deployment).** Mục tiêu thứ tư giải quyết bài toán phân phối và triển khai phần mềm (Software Deployment). Khác với các nghiên cứu chỉ để mô hình chạy trong môi trường cục bộ (localhost), dự án này yêu cầu toàn bộ kiến trúc phải được đưa lên môi trường Internet, phục vụ người dùng 24/7. Ứng dụng giao diện người dùng phải được phát triển bằng thư viện **Streamlit**, cung cấp các bảng điều khiển phân tích số liệu đồ họa tương tác (Interactive Analytics Dashboards) bằng Plotly. Nền tảng **Hugging Face Spaces** được lựa chọn làm môi trường triển khai dạng container (Containerized Host). Ứng dụng phải có khả năng khởi động tự động, kéo tệp tin cơ sở dữ liệu ZIP trực tiếp từ Data Lake, giải nén vào bộ nhớ cục bộ (RAM/Disk) và mở kết nối hệ thống trong vòng chưa tới 15 giây (Cold-start time), tạo ra một trải nghiệm người dùng cuối (End-user UX) ổn định và trực quan, hỗ trợ tìm kiếm bằng ngôn ngữ tự nhiên và tên phim cụ thể thay vì mã ID kỹ thuật số.

**5. Đạt được chi phí vận hành tối thiểu (Zero-cost Infrastructure) trong thực tế.** Mục tiêu cuối cùng, và cũng là mục tiêu mang tính thách thức nhất về mặt tư duy thiết kế, là tối ưu hóa nền tảng tài chính của hệ thống. Kiến trúc đề tài phải chứng minh được tính khả thi trong việc cung cấp một hệ thống Big Data MLOps cấp doanh nghiệp mà không cần tiêu tốn ngân sách duy trì hàng tháng. Việc này đòi hỏi sự tích hợp khéo léo giữa các nhà cung cấp dịch vụ đám mây có chính sách miễn phí phù hợp. Cloudflare R2 phải được sử dụng làm Data Lake để triệt tiêu hoàn toàn chi phí băng thông tải ra (Egress Fees) khi Streamlit liên tục kéo tập dữ liệu *ratings.csv* lên tới 500.000 dòng. Hugging Face Spaces cung cấp phần cứng miễn phí, và Groq API cung cấp giới hạn gọi API không tính phí. Thiết kế ứng dụng phải bao gồm các cơ chế Dự phòng Thông minh (Smart Fallback) để đảm bảo rằng, ngay cả khi API của bên thứ ba bị vượt quá hạn mức (Quota Limit Exhausted), hệ thống vẫn tự động chuyển về trạng thái hoạt động ngoại tuyến cục bộ (Offline Local Mode) thông qua LanceDB mà không gây gián đoạn toàn hệ thống (System Crash).

Việc hoàn thành 5 mục tiêu khắt khe này không chỉ khẳng định năng lực kỹ thuật trong việc thao tác với dữ liệu lớn và AI tạo sinh, mà còn thể hiện tư duy kiến trúc sư hệ thống (Systems Architect) trong việc giải quyết các bài toán về tối ưu hóa tài nguyên và tính khả dụng của phần mềm trong kỷ nguyên đám mây.

## 1.4 Phạm vi và giới hạn đề tài

Để đảm bảo đề tài được thực hiện một cách chuyên sâu, giải quyết trọn vẹn bài toán đặt ra mà không bị phân tán tài nguyên vào các yếu tố ngoại vi, phạm vi và các giới hạn kỹ thuật của

dự án được xác định một cách rõ ràng và khắt khe. Các giới hạn này phản ánh sự cân bằng giữa tham vọng học thuật và tính khả thi trong việc triển khai thực tế.

**Về phạm vi bộ dữ liệu (Dataset Scope):** Hệ thống sử dụng duy nhất một nguồn dữ liệu gốc (Single Source of Truth) là tập dữ liệu học thuật mở **MovieLens 25M**. Tập dữ liệu này chứa đầy đủ các bảng dữ liệu liên quan đến thực thể phim (*movies.csv*), lịch sử đánh giá (*ratings.csv*), hệ thống nhãn dán (*tags.csv*) và điểm tương quan hệ gen (*genome-scores.csv*). Đề tài giới hạn không mở rộng sang các tập dữ liệu khác như IMDb hay Netflix Prize nhằm đảm bảo sự tập trung vào việc tối ưu hóa đường ống ETL nội bộ. Đồng thời, đề tài không bao gồm việc phát triển các mô-đun cào dữ liệu tự động (Web Scraping) hay xây dựng các đường ống thu nhận dữ liệu người dùng mới theo thời gian thực (Real-time Event Ingestion Pipeline) từ các hệ thống Frontend bên ngoài. Mọi chỉ số phân tích, gợi ý và nội suy đều dựa trên một khối lượng kiến thức tĩnh đã được chốt sổ (Snapshot) từ kho lưu trữ của MovieLens.

**Về phạm vi nền tảng kiến trúc (Platform Scope):** Kiến trúc hệ thống được đóng khung nghiêm ngặt trong hệ sinh thái của ba nhà cung cấp dịch vụ đám mây chính: **Google Colab** (phục vụ mục đích tính toán dữ liệu lớn, tiền xử lý và nhúng vector bằng GPU/TPU tạm thời), **Hugging Face Spaces** (phục vụ mục đích triển khai vùng chứa Docker, cung cấp giao diện người dùng và xử lý các luồng logic Backend bằng CPU), và **Cloudflare R2** (đóng vai trò là Object Storage Data Lake chuyên biệt để lưu trữ các tệp tin CSV thô và các tệp nén cơ sở dữ liệu Vector ZIP). Đề tài không đặt trọng tâm vào việc cấu hình, cài đặt và duy trì các máy chủ vật lý, máy chủ ảo tính trên AWS/Azure (EC2/VMs), các cụm máy tính nội bộ (On-premise Clusters) hay việc triển khai các cơ sở dữ liệu quan hệ phức tạp (như PostgreSQL, MySQL).

**Về giới hạn kỹ thuật mô hình AI (Technical Limitations):** Hệ thống tích hợp Tác tử AI thông qua việc sử dụng Mô hình Ngôn ngữ Lớn (LLM) **Llama-3.3-70b-versatile**. Tuy nhiên, đề tài chỉ sử dụng mô hình này dưới dạng "Hộp đen" (Black-box) thông qua việc gọi giao diện lập trình ứng dụng (API) được cung cấp sẵn bởi nền tảng Groq [19]. Đề tài **không thực hiện quá trình tinh chỉnh vi mô (Fine-tuning)** hay huấn luyện lại các lớp mạng nơ-ron sâu bên trong LLM bằng tập dữ liệu MovieLens. Sức mạnh của AI trong hệ thống này hoàn toàn phụ thuộc vào năng lực Kỹ thuật Kích hoạt (Prompt Engineering), Kỹ thuật Gọi hàm (Function Calling) và độ chính xác của cơ sở dữ liệu cung cấp qua kiến trúc RAG. Đối với mô hình nhúng (Embedding Model), dự án chỉ sử dụng phiên bản trọng số đã được huấn luyện trước (Pre-trained weights) của **SentenceTransformer (all-MiniLM-L6-v2)**, không tiến hành huấn luyện độ đo khoảng cách tương phản (Contrastive Learning) trên dữ liệu đặc thù.

**Về giới hạn triển khai và vận hành (Deployment Limitations):** Do tôn chỉ của dự án là tối ưu hóa chi phí về mức 0, tính khả dụng và hiệu năng tối đa của hệ thống phụ thuộc hoàn toàn vào các **Hạn mức Dịch vụ Miễn phí (Free Tier Quotas)** của các nhà cung cấp bên thứ ba. Cụ thể, tốc độ phản hồi và khả năng hoạt động của Trợ lý AI bị ràng buộc cứng bởi Giới hạn Số lượng Token mỗi phút (TPM - Tokens Per Minute) và Số yêu cầu mỗi phút (RPM - Requests Per Minute) của Groq API. Trong trường hợp hệ thống có lượng truy cập tăng vọt đột biến (Traffic Spikes) vượt quá giới hạn của tài khoản miễn phí, chức năng tạo văn bản ngôn ngữ tự nhiên sẽ bị từ chối dịch vụ. Mặc dù hệ thống đã có cơ chế Dự phòng Thông minh (Smart Fallback) để chuyển sang dùng cơ sở dữ liệu tĩnh, nhưng chất lượng giao tiếp sẽ bị suy giảm thành các luồng gợi ý văn bản đơn giản. Thêm vào đó, việc phân tích biểu đồ trên Hugging Face Spaces bị giới hạn dung lượng tải về từ R2 (tối đa 500.000 dòng dữ liệu *ratings.csv*) để tránh lỗi tràn bộ nhớ (Out-of-Memory), điều này tạo ra giới hạn về khả năng hiển thị bức tranh toàn cảnh 100% của tập dữ liệu trên Giao diện phân tích thời gian thực.

Những giới hạn này không làm suy giảm giá trị khoa học của đề tài, mà ngược lại, chúng

định hình một lăng kính thực tế (pragmatic view) cho kỹ sư hệ thống. Việc nhận thức rõ ràng những giới hạn của công nghệ đám mây miễn phí và công nghệ học máy hộp đen giúp dự án thiết kế được những cơ chế dự phòng (fail-safes) xuất sắc, đảm bảo sản phẩm phần mềm luôn hoạt động ổn định và nhất quán trong một giới hạn kiểm soát nghiêm ngặt.

## 1.5 Cấu trúc báo cáo

Báo cáo đồ án được trình bày một cách logic, mạch lạc, đi từ cơ sở lý thuyết nền tảng đến việc ứng dụng các giải pháp kỹ thuật, và cuối cùng là quá trình triển khai, kiểm thử trên môi trường thực tế. Ngoài phần tổng quan, nội dung cốt lõi của báo cáo được cấu trúc thành 5 phần tiếp theo như sau:

**Phần 2 - Khảo sát các Công trình Liên quan và Cơ sở Lý thuyết:** Phần này cung cấp nền tảng kiến thức chuyên môn vững chắc, phân tích và so sánh các phương pháp luận cốt lõi trong lĩnh vực Hệ thống Gợi ý, từ Lọc Cộng tác (Collaborative Filtering), Lọc dựa trên Nội dung (Content-based), cho đến các hệ thống Lai (Hybrid). Phần này cũng tổng hợp các nghiên cứu mới nhất về công nghệ Cơ sở dữ liệu Vector (như FAISS, Pinecone, LanceDB), vai trò của các Mô hình Ngôn ngữ Lớn (LLMs), và sự dịch chuyển kiến trúc triển khai phần mềm theo tư duy MLOps hiện đại.

**Phần 3 - Phát biểu Bài toán và Thách thức Kỹ thuật:** Phần này định hình chi tiết các ràng buộc về đầu vào, đầu ra và hiệu năng của hệ thống. Đây là nơi phân tích các rào cản kỹ thuật phát sinh trong quá trình thực thi dự án, bao gồm lỗi tràn bộ nhớ RAM khi xử lý Big Data, lỗi cú pháp phân tích SQL của DataFusion, xung đột phụ thuộc phức tạp (Dependency Hell) giữa các thư viện Python, và bài toán ngăn chặn hiện tượng Ảo giác (Hallucination) của Trí tuệ nhân tạo.

**Phần 4 - Kiến trúc Hệ thống và Triển khai Kỹ thuật Hiện tại:** Phần này là phần trọng tâm của báo cáo, vẽ ra sơ đồ luồng dữ liệu toàn cảnh của hệ thống Serverless Cloud-native. Nội dung đi sâu phân tích đường ống tiền xử lý dữ liệu ETL trên Google Colab, kỹ thuật tối ưu hóa metadata Pandas trên Hugging Face Backend, kiến trúc cá nhân hóa giả lập tháp đôi (Pseudo-Tower Personalization), và đặc biệt là sự kết hợp tinh tế giữa quy trình RAG và cơ chế Gọi hàm (Function Calling) của Tác tử AI Llama-3.3-70b.

**Phần 5 - Đánh giá Hiệu năng và Kết quả Thực nghiệm:** Phần này cung cấp các bằng chứng định lượng và định tính về kết quả của dự án. Hệ thống được đánh giá thông qua các chỉ số đo lường học máy tiêu chuẩn như Recall@10, NDCG@10, được trích xuất trực tiếp từ các phiên theo dõi (Tracking Sessions) trên nền tảng DagsHub/MLflow. Đồng thời, phần này cũng phân tích độ trễ phản hồi (Latency) của truy vấn Vector, giới hạn API của Groq, và hiệu quả chống lỗi tràn bộ nhớ trên giao diện biểu đồ tương tác của Streamlit.

**Phần 6 - Tổng kết và Hướng phát triển Tương lai:** Phần cuối cùng đúc kết lại toàn bộ những đóng góp về mặt kỹ thuật và giá trị thực tiễn mà đề tài đã đạt được. Từ những thành công và cả những hạn chế còn tồn đọng, phần này vạch ra các định hướng mở rộng tiềm năng trong tương lai, như việc tích hợp khả năng thu thập dữ liệu thời gian thực bằng Apache Kafka, mở rộng thuật toán lượng tử hóa mảng tích (IVF-PQ) cho hàng tỷ vector, hoặc tự tinh chỉnh (fine-tune) một mô hình LLM chuyên biệt nguồn mở để loại bỏ hoàn toàn sự phụ thuộc vào các API của bên thứ ba.

## 2 Các công trình liên quan và nền tảng lý thuyết

Để xây dựng một hệ thống gợi ý phim hiện đại, xử lý khối lượng dữ liệu rất lớn (Big Data) và vận hành với độ trễ thấp, việc nghiên cứu các công trình liên quan và nắm vững cơ sở lý thuyết là cần thiết. Phần này trình bày một cách hệ thống về sự tiến hóa của các phương pháp tiếp cận trong lĩnh vực Hệ thống Gợi ý (Recommender Systems), công nghệ nền tảng của Cơ sở dữ liệu không gian Vector (Vector Database), và sự phát triển của Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) trong hệ thống hội thoại. Đồng thời, phần này cũng đi sâu vào việc so sánh các khung triển khai hệ thống máy học (MLOps Frameworks) để làm rõ mức độ phù hợp của kiến trúc Serverless Cloud-native được ứng dụng trong đề án.

### 2.1 Hệ thống gợi ý (Recommendation Systems)

Hệ thống gợi ý đã trở thành một thành phần cốt lõi không thể thiếu đối với mọi nền tảng kỹ thuật số hiện đại. Sự thành công của các tập đoàn công nghệ lớn như Netflix, Amazon hay YouTube phụ thuộc rất lớn vào khả năng của họ trong việc dự đoán chính xác nhu cầu của người dùng trước cả khi người dùng nhận ra điều đó. Các phương pháp gợi ý truyền thống được phân chia thành ba nhóm chính, mỗi nhóm đều mang những ưu điểm và hạn chế mang tính hệ thống.

#### 2.1.1 Collaborative Filtering (Lọc Cộng tác)

Lọc cộng tác (Collaborative Filtering - CF) là một trong những kỹ thuật nền tảng và thành công nhất trong lịch sử phát triển của hệ thống gợi ý [1]. Nguyên lý cốt lõi của CF không dựa trên việc hệ thống hiểu được nội dung của sản phẩm (như phim thuộc thể loại gì, diễn viên là ai), mà dựa trên việc khai thác hiệu quả của "trí tuệ đám đông" (wisdom of the crowd). Cụ thể, CF hoạt động theo hai luồng tư duy chính: *User-Based* (Lọc dựa trên người dùng) và *Item-Based* (Lọc dựa trên sản phẩm). Trong phương pháp User-Based, hệ thống tìm kiếm những người dùng có lịch sử đánh giá hoặc hành vi tương đồng với người dùng mục tiêu, từ đó đề xuất những sản phẩm mà nhóm "láng giềng" (neighbors) này đã yêu thích. Ngược lại, phương pháp Item-Based (do Amazon phổ biến) tính toán độ tương đồng trực tiếp giữa các sản phẩm dựa trên việc chúng có thường xuyên được đánh giá cùng nhau bởi cùng một nhóm người dùng hay không. Phương pháp Item-Based thường mang lại độ ổn định cao hơn do bản chất của các sản phẩm thường ít thay đổi so với sở thích biến thiên của con người.

Để giải quyết bài toán tính toán trên ma trận tương tác (User-Item Interaction Matrix), kỹ thuật Phân rã Ma trận (Matrix Factorization) đã ra đời và tạo nên tiếng vang lớn sau cuộc thi Netflix Prize [2]. Trong thực tế, một hệ thống có 100.000 người dùng và 10.000 bộ phim sẽ tạo ra một ma trận 1 tỷ ô, nhưng chỉ có chưa tới 1% số ô có chứa dữ liệu đánh giá thực sự (tính thưa thớt - sparsity). Phân rã ma trận, tiêu biểu là các thuật toán như SVD (Singular Value Decomposition) hay ALS (Alternating Least Squares), giải quyết sự thưa thớt này bằng cách nén (compress) cả người dùng và sản phẩm vào một **Không gian Ẩn (Latent Space)** có số chiều thấp hơn rất nhiều (ví dụ: 50 hoặc 100 chiều). Mỗi chiều trong không gian ẩn này đại diện cho một đặc trưng trừu tượng (như mức độ phim mang yếu tố hài hước, hoặc mức độ phim thiên về hành động) được mô hình tự động học ra. Điểm số dự đoán (predicted rating) của một người dùng đối với một bộ phim đơn giản chính là tích vô hướng (dot product) của hai vector đại diện của họ trong không gian ẩn này. Việc sử dụng ALS đặc biệt hữu hiệu trong việc huấn luyện song song (parallel training) trên các cụm máy chủ Spark dành cho dữ liệu lớn.

Ưu điểm chính của Collaborative Filtering là khả năng hoạt động độc lập (Domain agnostic)

với nội dung của sản phẩm. Hệ thống không cần một thuật toán xử lý ngôn ngữ tự nhiên (NLP) phức tạp hay các quy trình gán nhãn thủ công tốn kém để biết phim nói về gì. Miễn là có đủ dữ liệu tương tác từ cộng đồng, CF có thể đưa ra những gợi ý mang tính khám phá cao (serendipity), giúp người dùng tìm thấy những bộ phim phù hợp mà họ chưa từng nghĩ đến, vượt ra khỏi ranh giới thể loại quen thuộc. Điều này tạo ra một vòng lặp gắn kết (engagement loop) mạnh mẽ. Trong các hệ thống thương mại điện tử, đây là yếu tố quan trọng để kích thích nhu cầu mua sắm chéo (cross-selling).

Tuy nhiên, giới hạn quan trọng của Collaborative Filtering nằm ở **Bài toán Khởi động Nguội (Cold-start Problem)**. Đối với một bộ phim mới được đưa lên nền tảng, nó chưa có bất kỳ lượt đánh giá nào, do đó gần như "vô hình" đối với thuật toán CF. Tương tự, một người dùng mới đăng ký hệ thống, chưa từng bấm xem bất kỳ bộ phim nào, sẽ không thể nhận được bất kỳ gợi ý cá nhân hóa nào ngoài danh sách các phim phổ biến nhất (Top Trending). Hơn thế nữa, CF là một hệ thống tính toán ma trận số học thuần túy, nên không phù hợp để hiểu trực tiếp các truy vấn bằng ngôn ngữ tự nhiên của con người. Nếu một người dùng đặt câu hỏi "*Hãy gợi ý cho tôi một bộ phim trinh thám lấy bối cảnh chiến tranh lạnh*", một mô hình Matrix Factorization sẽ khó trả lời chính xác, bởi vì nó không hiểu "trinh thám" hay "chiến tranh lạnh" là gì. Đây chính là lý do khiến kiến trúc đồ án này phải tìm kiếm một hướng đi hiện đại hơn bằng công nghệ Vector và LLM.

### 2.1.2 Content-Based Filtering (Lọc dựa trên Nội dung)

Để bù đắp những khiếm khuyết của Lọc cộng tác, hệ thống Lọc dựa trên Nội dung (Content-Based Filtering - CBF) tiếp cận bài toán từ góc nhìn dựa trên thông tin mô tả của chính bộ phim. Nguyên lý cốt lõi của CBF là phân tích trực tiếp các thuộc tính metadata của sản phẩm. Một "hồ sơ sản phẩm" (Item Profile) sẽ được xây dựng dựa trên tập hợp các thuộc tính vật lý hoặc ngữ nghĩa như: đạo diễn, dàn diễn viên, năm phát hành, thể loại phim, và quan trọng nhất là đoạn văn bản tóm tắt nội dung cốt truyện (Overview). Đồng thời, hệ thống cũng xây dựng một "hồ sơ người dùng" (User Profile) thông qua việc tổng hợp (aggregation) các thuộc tính của những bộ phim mà người dùng đó đã từng xem và đánh giá cao. Ví dụ, nếu người dùng thường xuyên cho 5 sao các bộ phim do Christopher Nolan đạo diễn và có thể loại "Sci-Fi", hồ sơ của họ sẽ được gán trọng số cao cho hai thuộc tính này. Hệ thống gợi ý về bản chất là phép toán tính độ tương đồng giữa User Profile và Item Profile.

Trong các hệ thống CBF cổ điển, kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) phổ biến nhất để chuyển đổi văn bản mô tả phim thành định dạng máy tính có thể hiểu được là thuật toán **TF-IDF** (Term Frequency-Inverse Document Frequency) kết hợp với mô hình Túi từ (Bag of Words). TF-IDF đánh giá tầm quan trọng của một từ vựng dựa trên tần suất nó xuất hiện trong một bộ phim cụ thể (TF) so với mức độ quý hiếm của từ đó trên toàn bộ tập dữ liệu phim (IDF). Những từ phổ biến như "người", "và", "là" (stop words) sẽ bị triệt tiêu trọng số, trong khi những từ mang tính đặc trưng cao như "ma trận", "robot", "phép thuật" sẽ được gán trọng số lớn. Mỗi bộ phim sau đó sẽ được biểu diễn thành một vector thưa thớt (sparse vector) với số chiều bằng đúng kích thước của bộ từ điển (vocabulary size). Khoảng cách Cosine giữa các vector này sẽ quyết định độ tương đồng giữa hai bộ phim.

Mặc dù giải quyết tốt bài toán Khởi động nguội đối với phim mới (chỉ cần có văn bản mô tả là hệ thống có thể gợi ý được ngay), CBF cổ điển lại vướng phải những hạn chế đáng kể về mặt ngữ nghĩa học. Việc mã hóa bằng TF-IDF không có khả năng nắm bắt được **ngữ nghĩa sâu (Deep Semantics)** và ngữ cảnh của câu. Hai bộ phim có mô tả khác nhau về mặt từ vựng (ví dụ một phim dùng từ "sát thủ", phim kia dùng từ "kẻ giết người") có thể bị TF-IDF coi là hai vector trực giao, dù chúng có nội dung tương đồng. Ngoài ra, chất lượng của CBF phụ



thuộc mạnh vào chất lượng của metadata; nếu mô tả phim quá ngắn hoặc bị sai lệch, kết quả gợi ý sẽ kém chính xác. Cuối cùng, hệ thống CBF thường tạo ra hiện tượng "chuyên biệt hóa quá mức" (Over-specialization), hay còn gọi là "bong bóng lọc" (Filter Bubble). Người dùng có thể liên tục nhận được những bộ phim rất giống nhau về thể loại, làm giảm cơ hội khám phá các dòng phim mới và khiến trải nghiệm ứng dụng kém đa dạng.

### 2.1.3 Hybrid Approach và Neural Collaborative Filtering

Sự phát triển của ngành công nghiệp hệ thống gợi ý đã dẫn đến việc kết hợp các phương pháp lại với nhau tạo thành Hệ thống Lai (Hybrid Approach), nhằm lấy ưu điểm của CF bù đắp cho nhược điểm của CBF và ngược lại. Một cột mốc quan trọng là việc áp dụng các mạng Nơ-ron nhân tạo học sâu (Deep Learning) vào bài toán gợi ý, hình thành kỹ thuật **Neural Collaborative Filtering (NCF)** [3]. NCF thay thế phép nhân vô hướng (dot product) tuyến tính của Matrix Factorization bằng một mạng Nơ-ron đa tầng (Multi-Layer Perceptron - MLP), cho phép mô hình học được các hàm phi tuyến tính phức tạp để nắm bắt các tương tác giữa người dùng và sản phẩm ở cấp độ cao. Kế thừa NCF, mô hình Wide & Deep của Google kết hợp một mạng tuyến tính (Wide - ghi nhớ quy luật cụ thể) và một mạng sâu (Deep - tổng quát hóa đặc trưng), tạo ra hiệu quả dự đoán cao cho các ứng dụng như Google Play Store.

Một kiến trúc Deep Learning quan trọng trong hệ thống gợi ý quy mô lớn là mô hình **Tháp đôi (Two-Tower Model)** [4]. Kiến trúc này bao gồm hai mạng Nơ-ron hoạt động song song: một User Tower để trích xuất đặc trưng lịch sử của người dùng, và một Item Tower để trích xuất đặc trưng nội dung (văn bản, hình ảnh) của sản phẩm. Hai tháp này ánh xạ kết quả của chúng vào cùng một không gian vector (Shared Embedding Space). Điểm mạnh của Two-Tower là khả năng tính toán trước (pre-compute) vector của hàng triệu sản phẩm và lưu vào cơ sở dữ liệu Vector. Ở thời điểm phục vụ theo thời gian thực (real-time serving), hệ thống chỉ cần chạy dữ liệu người dùng qua User Tower một lần để lấy Vector Người dùng, sau đó thực hiện tìm kiếm Láng giềng gần nhất (ANN Search) thay vì phải tính toán độ tương đồng với toàn bộ cơ sở dữ liệu.

Tuy nhiên, việc triển khai một kiến trúc Two-Tower đầy đủ đòi hỏi nguồn lực lớn. Nó yêu cầu phải có các cụm máy chủ GPU hoạt động 24/7 để liên tục huấn luyện (train) lại tháp User khi có hành vi mới, và một máy chủ suy luận (Inference Server) thường trực để phục vụ API. Điều này không phù hợp với triết lý kiến trúc *Serverless Cloud-native* (tối thiểu hóa chi phí) mà đề án này theo đuổi. Để giải quyết mâu thuẫn này, đề tài đề xuất mẫu thiết kế **Pseudo-Tower Personalization** (Cá nhân hóa giả lập tháp đôi). Thay vì huấn luyện một mạng Nơ-ron sâu phức tạp cho tháp người dùng, hệ thống sử dụng một phương pháp nội suy toán học không cần huấn luyện (training-free). Hệ thống trích xuất tính toán toàn bộ vector của các bộ phim từ cơ sở dữ liệu LanceDB, sau đó tính toán *Trung bình có trọng số (Weighted Average)* dựa trên lịch sử đánh giá của người dùng. Kỹ thuật này xấp xỉ vị trí của người dùng trong không gian đa chiều (tương tự chức năng của User Tower), cung cấp khả năng cá nhân hóa linh hoạt, cập nhật theo thời gian thực mà không phát sinh chi phí duy trì máy chủ Inference riêng.

## 2.2 Semantic Search và Vector Embeddings

Khi bài toán gợi ý được ánh xạ thành bài toán tìm kiếm vị trí hình học trong không gian đa chiều, trọng tâm kỹ thuật chuyển sang Nhúng Vector (Vector Embeddings) và Tìm kiếm Ngữ nghĩa (Semantic Search). Đây là bước chuyển từ tìm kiếm từ khóa cổ điển sang tìm kiếm dựa trên biểu diễn ngữ nghĩa.

### 2.2.1 Transformer-based Embeddings

Lịch sử của việc chuyển đổi ngôn ngữ tự nhiên thành vector bắt đầu bằng các mô hình nhúng từ vừng (word embeddings) như Word2Vec hay GloVe. Tuy nhiên, các mô hình này không có khả năng hiểu ngữ cảnh (context-independent). Từ "bank" trong "bờ sông" (river bank) và "ngân hàng" (bank account) bị chúng gộp chung thành một vector duy nhất. Sự ra đời của kiến trúc **Transformer** (đặc biệt là mô hình BERT - Bidirectional Encoder Representations from Transformers) vào năm 2018 tạo ra bước tiến quan trọng cho biểu diễn ngôn ngữ [7, 8]. Bằng cơ chế Tự Chú ý (Self-Attention), BERT có khả năng đọc văn bản theo cả hai chiều (trái sang phải và phải sang trái), giúp nó hiểu được ý nghĩa của một từ dựa trên toàn bộ các từ xung quanh. Tuy nhiên, BERT không được thiết kế tối ưu cho bài toán tính khoảng cách câu (sentence similarity). Nếu muốn so sánh 10.000 câu bằng BERT, hệ thống phải thực hiện hàng triệu phép suy luận mạng nơ-ron qua lại, rất tốn thời gian.

Để giải quyết vấn đề này, kiến trúc **Sentence-BERT (SBERT)** đã được phát triển bằng cách sử dụng cấu trúc mạng Nơ-ron Siêm (Siamese Network) [9]. SBERT thêm một lớp gộp (Pooling Layer - thường là Mean Pooling) vào đầu ra của BERT để nén toàn bộ thông tin của một câu văn bản dài thành một vector cố định duy nhất (fixed-size sentence embedding). Hai vector của hai câu khác nhau sau đó có thể được so sánh với nhau trực tiếp bằng phép đo Khoảng cách Cosine (Cosine Similarity). Trong khuôn khổ của đề tài, mô hình **all-MiniLM-L6-v2** đã được lựa chọn cẩn thận làm lõi động cơ mã hóa [10]. Đây là một phiên bản mô hình Transformer đã được chưng cất (distilled) để thu nhỏ kích thước chỉ còn vài chục Megabytes, nhưng vẫn giữ được 90% hiệu quả của mô hình BERT gốc. Với kích thước không gian vector đầu ra là **384 chiều**, mô hình tạo ra sự cân bằng phù hợp: đủ sâu để nắm bắt ngữ nghĩa phức tạp của các bộ phim, nhưng cũng đủ mỏng nhẹ (lightweight) để có thể chạy ổn định trên bộ xử lý trung tâm (CPU) của giao diện Hugging Face Spaces mà không cần dùng đến GPU đắt đỏ.

Từ nguyên lý hoạt động của SBERT, dự án áp dụng kỹ thuật **Tăng cường Văn bản Nhúng (Text Embedding Enhancement)**. Thông thường, các nhà phát triển chỉ nổi tiêu đề phim (Title) và mô tả cốt truyện (Overview) rồi đưa cho mô hình mã hóa. Tuy nhiên, SBERT là mô hình xử lý ngôn ngữ, nên khó phân biệt một bộ phim được đánh giá cao với một bộ phim được đánh giá thấp nếu cả hai có cốt truyện giống nhau. Kỹ thuật Text Embedding Enhancement khắc phục điều này bằng cách chèn trực tiếp các thông số định lượng vào chuỗi văn bản đầu vào thông qua các mã thông báo ảo (virtual tokens). Việc hệ thống tự động gán thêm chuỗi *"[Quality] Rating: X.X/10"* vào cuối mỗi mô tả phim trước khi gọi hàm `encode()` giúp mô hình tiếp nhận thêm tín hiệu chất lượng. Khi mô hình đọc được một chuỗi mô tả hấp dẫn đi kèm với cụm từ *"Rating: 8.5/10"*, vector của phim có xu hướng dịch chuyển gần hơn tới nhóm các phim có tín hiệu đánh giá cao trong không gian 384 chiều. Ngược lại, một bộ phim với cụm từ *"Rating: 4.2/10"* sẽ có xu hướng nằm xa hơn nhóm này. Việc làm giàu văn bản trước khi nhúng cung cấp thêm đặc trưng phân loại (classification features) cho hệ thống tìm kiếm vector mà không cần xây dựng một lớp mạng nơ-ron phân loại riêng biệt, qua đó cải thiện chất lượng (Precision & Recall) của kết quả hiển thị cho người dùng.

### 2.2.2 Vector Database và Approximate Nearest Neighbor (ANN)

Khi đã sở hữu hàng triệu vector đại diện cho các bộ phim, bài toán tiếp theo là tra cứu chúng trong thời gian thực. Phương pháp truyền thống nhất là Tìm kiếm Xuyên suốt (Brute-force Exact Search), nghĩa là tính toán Khoảng cách L2 (Euclidean) hoặc Khoảng cách Cosine giữa vector truy vấn (Query Vector) với toàn bộ hàng trăm ngàn vector trong cơ sở dữ liệu. Độ phức tạp tính toán của thuật toán này là  $O(N)$ , tỷ lệ thuận tuyến tính với số lượng sản phẩm. Nếu áp dụng vào hệ thống gợi ý hàng triệu phim (như Netflix hay YouTube), thời gian phản



hồi (latency) có thể kéo dài lên đến vài giây hoặc thậm chí vài phút, làm suy giảm đáng kể trải nghiệm tương tác thời gian thực của người dùng. Để vượt qua giới hạn tài nguyên này, khoa học máy tính đã phát triển các thuật toán **Tìm kiếm Láng giềng gần nhất xấp xỉ (Approximate Nearest Neighbor - ANN)** [11]. Thuật toán ANN chấp nhận một mức sai số nhỏ (thường dưới 1%) để đổi lấy tốc độ truy vấn nhanh hơn nhiều lần.

Hai kỹ thuật tối ưu hóa ANN phổ biến nhất được tích hợp trong hệ thống là **IVF (Inverted File Index)** và **PQ (Product Quantization)**. Thuật toán IVF giải quyết bài toán  $O(N)$  bằng phương pháp "Chia để trị". Trong giai đoạn xây dựng chỉ mục (Index Building), không gian vector được phân hoạch thành hàng trăm cụm (clusters) thông qua thuật toán phân cụm K-Means. Mỗi cụm được đại diện bởi một khối tâm (Centroid). Khi có một vector truy vấn đi vào, hệ thống không so sánh nó với toàn bộ cơ sở dữ liệu. Thay vào đó, nó tính khoảng cách với các khối tâm, chọn ra một vài cụm gần nhất, và chỉ tiến hành tìm kiếm chi tiết bên trong các vùng Voronoi (Voronoi cells) đó. Việc này giúp giảm đáng kể chi phí tính toán khi quy mô dữ liệu tăng. Trong khi đó, thuật toán PQ giải quyết bài toán về bộ nhớ RAM (Memory constraint). Mỗi vector 384 chiều (mỗi chiều 4 bytes Float32) đòi hỏi 1.5KB dung lượng. PQ nén dữ liệu bằng cách cắt dọc vector thành nhiều phân đoạn nhỏ (sub-vectors) và mã hóa chúng thành một mã số nguyên ngắn (short bytes code) từ một bảng mã (Codebook). Việc áp dụng chỉ mục lai **IVF-PQ** cho phép hệ thống nén hàng chục triệu vector vào bộ nhớ nhỏ gọn và tìm kiếm với tốc độ phần nghìn giây. Mặc dù lượng tử hóa có thể gây mất mát thông tin (information loss), tỷ lệ thu hồi (Recall) vẫn có thể duy trì ở mức cao, thường trên 95%, thông qua việc tối ưu tham số `num_bits` và `num_sub_vectors`.

Hệ sinh thái mã nguồn mở hiện nay cung cấp nhiều giải pháp Hệ quản trị Cơ sở dữ liệu Vector (Vector Database) hàng đầu như **FAISS** của Meta AI, **Pinecone**, **Weaviate**, hay **Milvus**. Tuy nhiên, FAISS chỉ là một thư viện Python đơn thuần, thiếu tính bền vững của dữ liệu trên đĩa (disk persistence). Pinecone, Weaviate, và Milvus cung cấp một kiến trúc Client-Server quy mô lớn, phù hợp cho các dự án Enterprise, nhưng yêu cầu doanh nghiệp phải chi trả hàng ngàn đô la mỗi tháng cho các cụm máy chủ đám mây hoạt động liên tục (Always-on Compute Instances). Đứng trước ràng buộc khắt khe về triết lý thiết kế Zero-cost, đồ án đã quyết định chọn **LanceDB** làm thành phần lõi của hệ thống [16]. Khác biệt với toàn bộ phần còn lại, LanceDB là một cơ sở dữ liệu dạng nhúng (embedded database), chạy trực tiếp bên trong tiến trình ứng dụng Python (in-memory execution) mà không cần cài đặt thêm bất kỳ máy chủ cơ sở dữ liệu nền nào. Nó tương thích gốc (Python-native) với định dạng dữ liệu bộ nhớ dạng cột PyArrow, hỗ trợ toàn diện các chỉ mục nâng cao như IVF-PQ, và có khả năng đọc thẳng dữ liệu từ dịch vụ Cloudflare R2 (Serverless Storage). Sự gọn nhẹ và mạnh mẽ của LanceDB là yếu tố hỗ trợ hiện thực hóa kiến trúc Serverless 100% của toàn bộ đồ án này.

## 2.3 Large Language Models và RAG

Lĩnh vực Học máy và Hệ thống gợi ý đã chính thức bước sang một kỷ nguyên mới với sự xuất hiện của các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) thế hệ mới, tiêu biểu là dòng mô hình Llama của Meta AI. Các LLM không chỉ có khả năng sinh văn bản xuất sắc mà còn sở hữu năng lực suy luận logic (Reasoning) vượt xa các mô hình ngôn ngữ thế hệ trước, biến chúng từ một công cụ sinh chữ trở thành những Tác tử thông minh tự chủ (Intelligent Agents).

### 2.3.1 Function Calling trong LLM

Khái niệm Gọi Hàm (Function Calling - hay Tool Use) là một bước tiến mang tính quan trọng trong quá trình phát triển LLM. Trước đây, LLM chỉ là các hệ thống đóng (Closed Systems),

mọi câu trả lời của nó đều bị giới hạn bởi tập dữ liệu văn bản mà nó đã được huấn luyện (pre-training data) cho đến thời điểm đóng băng tri thức (knowledge cutoff date). Nó hoàn toàn bị cô lập khỏi thế giới thực, không thể lấy được thông tin hiện tại (như giá chứng khoán, thời tiết, hay cơ sở dữ liệu phim nội bộ). Cơ chế Function Calling khắc phục hạn chế này bằng cách biến LLM thành một trung tâm điều phối (Orchestrator). Trong thiết kế này, nhà phát triển sẽ cung cấp cho LLM một bộ Lược đồ JSON (JSON Schemas) mô tả chi tiết tên, chức năng và các tham số truyền vào của các hàm lập trình cục bộ (Python functions). Khi người dùng đặt câu hỏi, LLM sẽ phân tích ý định ngôn ngữ (Intent Classification). Nếu nó nhận thấy mình không đủ kiến thức để trả lời, hoặc cần thao tác với dữ liệu bên ngoài, nó sẽ ngừng sinh văn bản (halt generation). Thay vào đó, nó sẽ tạo ra một cấu trúc JSON hợp lệ chứa các tham số được trích xuất từ câu hỏi của người dùng và gửi về cho máy chủ ứng dụng Python. Python sẽ thực thi hàm tương ứng với các tham số đó, lấy kết quả, và trả ngược lại cho LLM để nó tổng hợp thành câu trả lời cuối cùng bằng ngôn ngữ tự nhiên.

Hệ thống trợ lý ảo AI (Movie Concierge) trong đề án này được kiến tạo xung quanh mô hình suy luận đa năng **Llama-3.3-70b-versatile** vận hành trên hệ thống máy chủ Groq. Groq sử dụng các bộ vi xử lý LPU (Language Processing Units) được thiết kế đặc thù cho kiến trúc Transformer, cho phép tốc độ sinh token lên tới con số hàng ngàn mỗi giây, tạo cảm giác phản hồi gần thời gian thực. Ứng dụng này tận dụng hiệu quả Llama-3.3 bằng cách trang bị cho mô hình **5 công cụ chuyên biệt (Tools)**. Tác tử này có quyền tự quyết định (Autonomous decision) sử dụng công cụ tìm kiếm ngữ nghĩa LanceDB khi người dùng hỏi về cốt truyện, công cụ tìm kiếm thập niên nếu có mốc thời gian, công cụ so sánh khi được cung cấp hai định danh phim, công cụ truy xuất phim thịnh hành (Trending), hoặc công cụ gợi ý tương tự dựa trên lịch sử (Personalization). Sự tự chủ của tác tử LLM mang lại trải nghiệm người dùng khác biệt so với các thanh tìm kiếm truyền thống, nơi thuật toán chủ yếu phản hồi theo truy vấn một chiều.

### 2.3.2 Retrieval-Augmented Generation (RAG)

Bất chấp hiệu quả từ bộ nhớ tham số rất lớn (Parametric Memory) của mình, hạn chế mang tính hệ thống lớn nhất của toàn bộ các LLM hiện tại là hiện tượng Ảo giác Trí tuệ Nhân tạo (AI Hallucination). Khi đối mặt với các truy vấn liên quan đến tri thức chuyên ngành (Domain-specific knowledge)—như tên đạo diễn, thông số độ trễ, hay số điểm đánh giá chính xác của một bộ phim ngách không có trong dữ liệu huấn luyện (long-tail movies)—LLM có xu hướng tự động sinh sai (fabricate) ra những thông tin nghe có vẻ rất hợp lý và trôi chảy nhưng lại hoàn toàn sai sự thật. Đối với một hệ thống gợi ý thương mại, việc đưa ra thông tin giả mạo sẽ làm suy giảm nghiêm trọng độ tin cậy của dịch vụ. Để giải quyết dứt điểm sự cố này, dự án ứng dụng kiến trúc mẫu thiết kế **RAG (Retrieval-Augmented Generation)**.

Kiến trúc RAG hoạt động dựa trên triết lý cung cấp căn cứ sự thật (Grounding Truth) [12]. Quá trình xử lý một truy vấn RAG diễn ra theo một quy trình ba bước không thể tách rời: **Truy xuất (Retrieval)** → **Bổ sung ngữ cảnh (Augmentation)** → **Sinh văn bản (Generation)**. Đầu tiên, ngay khi tiếp nhận câu hỏi của người dùng, hệ thống mã hóa nó thành vector và truy xuất (Retrieve) danh sách Top-K các bộ phim thực tế có liên quan nhất từ kho lưu trữ sự thật LanceDB. Tiếp theo, thay vì để LLM chỉ dựa trên tri thức nội tại, toàn bộ nội dung của các tài liệu vừa truy xuất (bao gồm tiêu đề, thể loại, điểm số, mô tả) sẽ được hệ thống Python định dạng (format) thành một khối văn bản dài và bổ sung (Augment) trực tiếp vào chỉ thị hệ thống ẩn (System Prompt). Bước cuối cùng, LLM **Llama-3.3-70B** sẽ đọc khối tài liệu nền tảng đó, sử dụng khả năng tổng hợp ngôn ngữ của mình để trích xuất thông tin, tóm tắt và sinh ra (Generate) câu trả lời cuối cùng bằng tiếng Việt. LLM được cảnh báo

nghiêm ngặt chỉ được phép sử dụng các sự kiện có trong khối System Prompt.

Ưu điểm chính của kiến trúc RAG so với phương pháp Tinh chỉnh vi mô (Fine-tuning) mô hình là chi phí phần cứng và tính linh hoạt. Việc tinh chỉnh một mô hình 70 tỷ tham số để dạy cho nó kiến thức của 25 triệu dòng dữ liệu đòi hỏi các cụm máy chủ đa GPU cấu hình rất lớn, tiêu tốn hàng chục nghìn đô la và mất hàng tuần để huấn luyện. Bất cập hơn nữa, mỗi khi có một bộ phim mới phát hành, mô hình lại phải được huấn luyện lại từ đầu để cập nhật thông tin. RAG giải quyết bài toán này theo hướng hiệu quả: mô hình ngôn ngữ Llama không cần lưu giữ thông tin bộ phim trong trọng số mạng nơ-ron của nó (Stateless). Toàn bộ tri thức thế giới được bảo tồn một cách an toàn và gọn nhẹ bên trong Cơ sở dữ liệu Vector. Việc cập nhật kiến thức hệ thống giờ đây chỉ đơn giản là thao tác chèn (insert) một hàng dữ liệu mới vào LanceDB, một hành động tốn chưa tới vài phần ngàn giây và không chạm đến mô hình học máy. Bằng cách kết hợp LanceDB với Groq LLM trong mô hình RAG, hệ thống đã xây dựng được một trợ lý AI có khả năng phản hồi tốc độ cao, giao tiếp đa lượt với khả năng ghi nhớ thực thể (Entity Memory), và duy trì độ chính xác thông tin dựa trên dữ liệu truy xuất.

## 2.4 MLOps và Serverless Deployment

Quy trình thao tác học máy (MLOps - Machine Learning Operations) là bước chuyển hóa quan trọng nhất để đưa các thử nghiệm khoa học dữ liệu rời rạc (Data Science Experiments) trở thành các sản phẩm phần mềm (Software Products) đáng tin cậy. Nếu chỉ dừng lại ở các đoạn mã chạy thử công trên Jupyter Notebook, hệ thống không thể đáp ứng được các yêu cầu về khả năng duy trì, tính chịu lỗi (Fault-tolerance), và tính tự động hóa khi đi vào sản xuất. MLOps thiết lập các tiêu chuẩn khắt khe cho toàn bộ vòng đời mô hình: từ quản lý mã nguồn, theo dõi siêu tham số thử nghiệm, đăng ký mô hình, cho đến giám sát hiệu suất ở tầng phục vụ ứng dụng. Trong dự án, vai trò trung tâm của MLOps được thực thi thông qua nền tảng mã nguồn mở **DagsHub/MLflow** [15]. Mỗi chu kỳ khởi chạy của đường ống dữ liệu trên Google Colab đều tự động ghi lại toàn bộ các thông số liên quan (như kích thước cơ sở dữ liệu tạo ra, thời gian nhúng), cùng các chỉ số đo lường hiệu năng cốt lõi là *Recall@10* và *NDCG@10*. Việc lưu vết tập trung và công khai các bảng số liệu thực nghiệm (Experiment Tracking) đảm bảo tính minh bạch học thuật (Transparency) và khả năng tái lập kết quả thí nghiệm (Reproducibility), loại trừ rủi ro mã nguồn phụ thuộc vào một cá nhân kỹ sư cụ thể.

Về phương diện triển khai cơ sở hạ tầng (Infrastructure Deployment), sự phát triển của điện toán đám mây nguyên bản (Cloud-native) đã thay đổi đáng kể các quy chuẩn công nghiệp. Các kiến trúc đóng gói mô hình tinh thông qua Docker Container và kết nối qua cổng API RESTful như framework **BentoML** từng là lựa chọn phổ biến. Tuy nhiên, kiến trúc truyền thống đó đòi hỏi việc cung cấp sẵn các máy chủ ảo (Virtual Private Servers - VPS) như AWS EC2 hoặc DigitalOcean. Các máy chủ ảo này tiêu tốn năng lượng và tài nguyên điện toán 24/7 kể cả khi không có bất kỳ khách hàng nào truy cập (Always-on instances), tạo ra gánh nặng tài chính rất lớn, đặc biệt đối với môi trường giáo dục và thử nghiệm nguyên mẫu (Prototyping).

Trước bối cảnh đó, định hướng kỹ thuật của đề án đặt trọng tâm vào kiến trúc **Serverless** (Phi máy chủ). Nền tảng ứng dụng máy học **Hugging Face Spaces** cung cấp các vùng chứa động (Dynamic Containers) [17]. Các vùng chứa này có khả năng tự động ngắt kết nối và giải phóng tài nguyên về trạng thái 0 (Scale-to-zero) khi không nhận được lưu lượng truy cập, qua đó giảm chi phí vận hành. Ngay khi có người dùng truy xuất, hệ thống sẽ khởi động lại theo cơ chế Cold start, kéo (stream) dữ liệu tính trực tiếp từ kho lưu trữ đối tượng **Cloudflare R2 Data Lake** (dịch vụ lưu trữ phi tập trung không tính phí bằng thông tải ra - Zero Egress Fees) thông qua thư viện **boto3** [18]. Cấu trúc không trạng thái (Stateless Architecture) này cho phép máy chủ khởi tạo giao diện **Streamlit** ổn định và an toàn. Việc chuyển đổi từ kiến

trúc BentoML + Docker nặng nề sang Hugging Face Spaces + Cloudflare R2 không chỉ là một sự thay đổi công nghệ, mà là một minh chứng cho hiệu quả của triết lý MLOps tối ưu: tạo ra các hệ thống ứng dụng Big Data quy mô lớn với ngân sách bảo trì tiệm cận mức 0, mở rộng khả năng tiếp cận công nghệ cho sinh viên, nhà nghiên cứu và cộng đồng mã nguồn mở.

### 3 Phát biểu bài toán và các thách thức

Quá trình chuyển đổi từ một mô hình học thuật mang tính lý thuyết sang một hệ thống phần mềm trí tuệ nhân tạo vận hành trong thực tế (Production-ready AI System) luôn tiềm ẩn nhiều rào cản phức tạp. Phần này định nghĩa một cách chặt chẽ bài toán mà hệ thống cần giải quyết, đồng thời phân tích các thách thức kỹ thuật và phi kỹ thuật đã phát sinh trong suốt vòng đời phát triển dự án. Cùng với việc làm rõ nguyên nhân gốc rễ của từng sự cố (từ giới hạn phần cứng đến xung đột thư viện), các giải pháp kỹ thuật tinh gọn, tuân thủ triết lý MLOps và kiến trúc điện toán đám mây cũng sẽ được trình bày chi tiết.

#### 3.1 Phát biểu bài toán chính thức

Trước khi thiết kế kiến trúc hệ thống, việc xác định rõ ràng các tham số đầu vào, đầu ra, và những ràng buộc về mặt hiệu năng là bước tiên quyết để định hình các quyết định kỹ thuật.

##### 3.1.1 Định nghĩa bài toán

Bài toán cốt lõi của đề tài là xây dựng một **Hệ thống Gợi ý Phim Tích hợp Tác tử Hội thoại (Conversational Recommender System)**, có khả năng hiểu được nhu cầu phức tạp của người dùng và cung cấp các đề xuất chính xác dựa trên tập dữ liệu quy mô lớn.

**Đầu vào (Input)** của hệ thống không phải là các mã lệnh cố định hay các trường nhập liệu tĩnh, mà là các câu truy vấn bằng ngôn ngữ tự nhiên (Natural Language Queries) không có cấu trúc cố định từ người dùng. Ví dụ, một người dùng có thể yêu cầu: *"Hãy gợi ý cho tôi một vài bộ phim hành động mang tính triết lý sâu sắc, phát hành trong thập niên 90"*. Dạng thông tin đầu vào này mang tính đa nghĩa cao, đòi hỏi hệ thống không chỉ trích xuất từ khóa, mà còn diễn giải được ngữ cảnh, sắc thái yêu cầu và mốc thời gian. Ngoài ra, đầu vào còn bao gồm một danh sách các mã phim do người dùng chủ động lựa chọn từ trước để thiết lập hồ sơ sở thích cá nhân ban đầu (Explicit Feedback).

**Đầu ra (Output)** mong đợi là một danh sách K bộ phim (Top-K) phù hợp nhất được truy xuất từ cơ sở dữ liệu. Khác biệt với các hệ thống gợi ý truyền thống chỉ trả về một danh sách các ảnh bìa phim thụ động, yêu cầu bắt buộc đối với đầu ra của hệ thống này là phải đi kèm với lời giải thích ngữ nghĩa rõ ràng. Mỗi bộ phim được gợi ý phải hiển thị đầy đủ tiêu đề, năm phát hành, điểm số trung bình thực tế, và một đoạn văn mạch lạc được diễn đạt thông qua Tác tử Trí tuệ Nhân tạo (AI Agent), nhằm giúp người dùng hiểu rõ tại sao bộ phim đó lại phù hợp với hồ sơ và truy vấn của họ.

Bên cạnh yếu tố chính xác, hệ thống phải tuân thủ các **Ràng buộc cứng (Hard Constraints)** khắt khe về mặt vận hành. Tính khả dụng của một hệ thống hội thoại phụ thuộc rất lớn vào độ trễ; do đó, thời gian phản hồi toàn trình (End-to-end latency) từ lúc nhận truy vấn đến lúc hiển thị kết quả lên màn hình phải **dưới 3 giây**. Quan trọng hơn, giải pháp kiến trúc phải thỏa mãn ràng buộc quan trọng nhất về mặt kinh tế: toàn bộ hệ thống phải chạy ổn định trên các nền tảng đám mây cung cấp gói miễn phí (free-tier cloud services), đưa chi phí duy trì và hạ tầng hệ thống về mức xấp xỉ 0. Về mặt **Ràng buộc mềm (Soft Constraints)**, kết quả đề xuất không được phép quá tập trung vào một thể loại duy nhất (tính đa dạng -

diversity) và phải phản ánh được sự biến thiên trong lịch sử xem phim của người dùng, tránh hiện tượng "bong bóng lọc" (filter bubble).

### 3.1.2 Các sub-task cần giải quyết

Để giải quyết trọn vẹn bài toán tổng thể đã nêu, đề án tiến hành phân rã hệ thống thành 5 bài toán con (sub-tasks) cụ thể, mỗi bài toán yêu cầu một công nghệ xử lý chuyên biệt:

1. **Hiểu ý định người dùng (Intent Classification):** Hệ thống phải xác định xem câu nói của người dùng thuộc loại yêu cầu gì (tìm phim mới, so sánh phim, tìm phim theo năm, hay chỉ là câu chào hỏi thông thường). Bài toán này được giải quyết thông qua cơ chế Gọi hàm (Function Calling) tích hợp bên trong Mô hình Ngôn ngữ Lớn, cho phép LLM tự động phân tích câu văn thành các tham số JSON có cấu trúc.
2. **Tìm kiếm ngữ nghĩa trong không gian vector lớn (Semantic Search):** Việc so sánh nội dung của hàng chục ngàn bộ phim trong vài mili-giây không thể thực hiện bằng SQL. Sub-task này yêu cầu hệ thống phải chuyển đổi mô tả phim thành các vector nhúng (embeddings) và thực thi thuật toán K-Láng giềng gần nhất xấp xỉ (ANN KNN) trên cơ sở dữ liệu không gian.
3. **Lọc metadata (Metadata Filtering):** Trong nhiều trường hợp, truy vấn của người dùng mang tính điều kiện cứng (ví dụ: "chỉ lấy phim thập niên 90" hoặc "chỉ lấy phim kinh dị"). Hệ thống cần một công cụ lọc nhanh chóng và ổn định trước hoặc sau khi tìm kiếm vector, mà không bị vướng vào các lỗi phân tích cú pháp bất ổn (unstable SQL builder) của các trình biên dịch nội bộ.
4. **Cá nhân hoá kết quả theo lịch sử người dùng (Personalization):** Đây là bài toán cốt lõi của Lọc Cộng tác. Hệ thống phải có khả năng nội suy ra sở thích ẩn của người dùng dựa trên những gì họ đã xem, và dùng nó làm trọng số để định hướng lại (bias) kết quả tìm kiếm vector, đảm bảo người dùng  $A$  và người dùng  $B$  sẽ nhận được kết quả khác nhau dù gõ cùng một từ khóa.
5. **Sinh ngôn ngữ tự nhiên giải thích kết quả (LLM Generation):** Khi đã có danh sách phim dạng thô từ cơ sở dữ liệu, bài toán cuối cùng là làm sao để máy tính "nói chuyện" được với con người. Sub-task này yêu cầu khả năng tổng hợp văn bản, tóm tắt cốt truyện và diễn đạt logic một cách tự nhiên, trôi chảy, sử dụng ngữ pháp tiếng Việt chuẩn xác thông qua mô hình tạo sinh.

## 3.2 Thách thức kỹ thuật

Trong quá trình thực thi dự án, việc tích hợp hàng loạt các công nghệ phân tán (Hugging Face, LanceDB, Groq LLM, Pandas) vào một đường ống thống nhất đã làm nảy sinh vô số rào cản kỹ thuật rất phức tạp. Dưới đây là 5 thách thức lớn nhất và cách hệ thống đã vượt qua chúng.

### 3.2.1 Thách thức 1: Quy mô dữ liệu vs. RAM giới hạn

**Mô tả vấn đề:** Tập dữ liệu MovieLens 25M chứa hơn 25 triệu bản ghi đánh giá (ratings) và hàng triệu nhãn dán (tags) từ cộng đồng [6]. Tổng dung lượng lưu trữ thô ở định dạng CSV lên tới hàng Gigabytes. Khi triển khai lớp Giao diện Bảng điều khiển phân tích (Analytics Dashboard) trên nền tảng Hugging Face Spaces (sử dụng Streamlit), máy chủ chỉ cung cấp mức tài nguyên miễn phí (free-tier) khá hạn chế, thường bị giới hạn cứng ở mức 16GB RAM, và có thể thấp tới 2GB ở các thời điểm tài nguyên dùng chung bị thắt chặt. Nếu sử dụng thư viện Pandas để tải (load) toàn bộ 25 triệu dòng dữ liệu vào bộ nhớ trong (In-memory) cùng



một lúc nhằm thao tác và vẽ các biểu đồ phân tích, vùng chứa (container) của ứng dụng sẽ dễ bị hệ điều hành đóng băng hoặc dừng hoạt động do lỗi tràn bộ nhớ (Out-Of-Memory - OOM).

**Chiến lược giải quyết:** Để vượt qua giới hạn tài nguyên này, hệ thống áp dụng chiến lược phân tách môi trường triệt để (Environment Decoupling). Quá trình tải, làm sạch và kết nối (join) toàn bộ 25 triệu dòng dữ liệu được đẩy hoàn toàn sang môi trường huấn luyện mạnh mẽ trên Google Colab (được cấp phát GPU và dung lượng RAM lớn hơn rất nhiều). Tại Colab, dữ liệu được chuyển đổi thành cơ sở dữ liệu Vector LanceDB tĩnh, sau đó nén chặt thành một tệp `lancedb_movies.zip` gọn nhẹ (chỉ khoảng vài chục Megabytes) để tải lên đám mây. Ở phía lớp phục vụ (Serving Layer) trên Hugging Face, ứng dụng không cần truy cập trực tiếp tập dữ liệu CSV rất lớn ban đầu, mà chỉ kéo tệp ZIP đã nén về để khởi tạo Động cơ Tìm kiếm Vector.

**Tối ưu hóa Bảng điều khiển (Analytics):** Đối với nhu cầu vẽ biểu đồ thống kê theo thời gian thực trên Streamlit, việc tải 25 triệu dòng là không khả thi. Do đó, hệ thống ứng dụng kỹ thuật Truyền phát Giới hạn (Capped Streaming). Hàm tải dữ liệu được thiết kế kết nối với Cloudflare R2 thông qua `boto3` nhưng sử dụng tham số `nrows=500000`. Việc chỉ đọc tối đa 500.000 dòng đầu tiên (hoặc ngẫu nhiên) tạo ra một sự đánh đổi (trade-off) tinh tế: chúng ta hy sinh một phần độ chính xác cao trên tập 25M, để đổi lấy một tập mẫu đại diện (representative sample) vừa đủ lớn để phản ánh đúng phân phối thống kê chuẩn của tập dữ liệu (xu hướng thập kỷ, phân bố rating), mà vẫn đảm bảo dung lượng tiêu thụ RAM của Hugging Face luôn an toàn dưới mức báo động đỏ.

### 3.2.2 Thách thức 2: Bất ổn của DataFusion SQL Parser

**Mô tả vấn đề:** LanceDB là một cơ sở dữ liệu vector nhúng hoạt động dựa trên lõi lập trình Rust, và nó sử dụng engine DataFusion (thuộc hệ sinh thái Apache Arrow) để thực thi các lệnh truy vấn phân tích (Analytics Queries) và lọc metadata (Metadata Filtering). Khi ứng dụng cần tìm kiếm chính xác một bộ phim thông qua định danh, cú pháp hàm chuẩn là sử dụng `table.search().where('movieId' = 1')`. Tuy nhiên, trong môi trường Hugging Face, bộ biên dịch SQL của DataFusion thường xuyên phát sinh lỗi ngoại lệ *LanceError(Schema): No field named movieid*.

**Phân tích nguyên nhân:** Sự cố này bắt nguồn từ việc bộ phân tích cú pháp (parser) của DataFusion không hỗ trợ đầy đủ và nhất quán các cú pháp bọc định danh (Identifier Enclosure) trong môi trường Python bị đóng gói. Nó tự động chuyển đổi tất cả các tên cột không được bọc chặt về định dạng chữ thường (lowercase normalization). Do lược đồ cơ sở dữ liệu gốc của chúng ta có phân biệt chữ hoa chữ thường (`movieId` với chữ 'I' viết hoa), DataFusion không tìm thấy cột `movieid` (chữ thường) trong lược đồ, khiến truy vấn bị vô hiệu hóa hoàn toàn, trả về kết quả rỗng và làm gián đoạn luồng gọi hàm của LLM.

**Giải pháp thay thế:** Nhận thấy cơ sở dữ liệu vector sau khi được làm sạch chỉ còn ở quy mô vài chục nghìn bản ghi, dung lượng tổng thể của bảng rất gọn nhẹ: notebook tạo bảng master 13,164 phim sau khi merge TMDb, còn dashboard phục vụ hiển thị 27,278 phim vectorized. Để giải quyết triệt để sự bất ổn định của DataFusion SQL parser, hệ thống áp dụng chiến lược **Pandas Filtering**. Cụ thể, thay vì yêu cầu LanceDB lọc metadata thông qua cú pháp `.where()`, ứng dụng gọi lệnh `table.to_pandas()` để chuyển toàn bộ bảng vector vào cấu trúc DataFrame của Pandas trong bộ nhớ RAM. Sau đó, việc tra cứu ID, lọc thập niên, hay lọc thể loại được thực hiện bằng các thao tác lập chỉ mục boolean (boolean indexing) thuần túy của Python (`df[df["movieId"] == movie_id]`). Kỹ thuật này loại bỏ hoàn toàn rủi ro liên quan đến bộ biên dịch SQL bên thứ ba, đảm bảo an toàn cao (100% robust) cho các thao tác lọc dữ liệu, trong khi tốc độ vẫn nằm trong ngưỡng vài phần nghìn giây.



### 3.2.3 Thách thức 3: Xung đột phiên bản thư viện

**Mô tả vấn đề:** Việc duy trì một hệ thống MLOps đòi hỏi sự đồng nhất cao về phiên bản các gói phần mềm (Packages) giữa môi trường phát triển (Training) và môi trường triển khai (Serving). Dự án này đã đối mặt với các xung đột phụ thuộc phức tạp (Dependency Hell) liên quan đến hai thư viện cốt lõi là Groq và LanceDB.

**Chi tiết xung đột:** Thứ nhất, bộ phát triển phần mềm (SDK) của Groq (dùng để gọi LLM Llama) phụ thuộc chặt chẽ vào thư viện giao tiếp mạng `httpx`. Gần đây, `httpx` phát hành bản cập nhật  $\geq 0.28.0$ , trong đó họ đổi tên tham số cấu hình từ `proxies` thành `proxy`. Tuy nhiên, mã nguồn bên trong SDK của Groq chưa được nhà phát hành cập nhật kịp thời, dẫn đến lỗi `TypeError: Client.__init__() got an unexpected keyword argument 'proxies'`, làm không thể khởi động ứng dụng Streamlit ngay khi khởi động. Thứ hai, trong môi trường Google Colab, thư viện `lancedb` bản mới nhất ( $\geq 0.26.0$ ) khi tạo chỉ mục lượng tử hóa mảng tích (IVF-PQ) đã tự động thêm trường metadata `num_bits` vào cấu trúc JSON của nó. Tuy nhiên, nếu trên Hugging Face cấu hình dùng phiên bản cũ hơn (ví dụ `0.22.0`), nó sẽ không hiểu trường `num_bits` này là gì, dẫn đến lỗi `LanceError(Arrow): missing field num_bits` và từ chối tải cơ sở dữ liệu.

**Bài học và Giải pháp:** Sự cố này chứng minh rằng việc quản lý phiên bản trong tệp `requirements.txt` là yếu tố quan trọng quyết định sự thành bại của hệ thống MLOps. Để khắc phục, hệ thống đã thực hiện ghim cứng (version pinning) các thư viện xung đột. Thư viện `httpx` được ràng buộc cài đặt ở phiên bản `httpx==0.27.2` để đảm bảo Groq SDK hoạt động trơn tru. Đồng thời, phiên bản của LanceDB trên Hugging Face được quy định cứng là `lancedb>=0.26.0` để duy trì khả năng đồng bộ metadata với tệp ZIP được nén từ Colab. Những bài học thực chiến này giúp dự án trở nên ít phụ thuộc vào các lỗi gián đoạn ứng dụng tiềm ẩn do nhà cung cấp cập nhật phần mềm.

### 3.2.4 Thách thức 4: Ngăn chặn Hallucination của LLM

**Mô tả vấn đề:** Mô hình **Llama-3.3-70B** là một Mô hình Ngôn ngữ Lớn sinh xác suất. Mặc dù sở hữu năng lực suy luận mạnh, nhược điểm quan trọng của nó—giống như mọi LLM khác—là hiện tượng Ảo giác (Hallucination). Khi được yêu cầu gợi ý phim trong các lĩnh vực chuyên biệt (domain-specific tasks), nếu chỉ dựa vào khối trọng số được huấn luyện từ trước, LLM rất dễ sinh ra những thông tin nghe có vẻ hợp lý nhưng sai sự thật. Nó có thể gán sai tên đạo diễn cho một bộ phim, tự động tạo ra một bộ phim hành động không hề tồn tại, hoặc tự ý nâng điểm đánh giá trung bình của một bộ phim kém chất lượng lên 9/10 để làm hài lòng người dùng.

**Giải pháp RAG thuần túy:** Để hạn chế hiện tượng ảo giác này, hệ thống áp dụng một mẫu thiết kế **RAG (Retrieval-Augmented Generation) nguyên bản** [12]. Nguyên tắc chính của thiết kế này là: LLM không được phép sử dụng trí nhớ tham số của chính nó để trả lời về thông tin phim ảnh. LLM chỉ được cấp phép sinh ngôn ngữ dựa trên dữ liệu thực tế được cung cấp cho nó.

**Cơ chế hoạt động:** Khi người dùng đặt câu hỏi, mã nguồn Python sẽ bẻ luồng truy vấn và trực tiếp đi tìm kiếm trong cơ sở dữ liệu LanceDB. Danh sách Top-K các bộ phim phù hợp nhất sẽ được LanceDB trả về. Toàn bộ thông tin cấu trúc của các bộ phim này—bao gồm tiêu đề chính xác, năm phát hành, thể loại, điểm số thập phân, và tóm tắt nội dung—được mã nguồn Python nối thành một khối văn bản dài. Khối văn bản này được tiêm (inject) thẳng vào **System Prompt** của Groq trước khi LLM bắt đầu sinh từ. Thông qua kỹ thuật Prompt Engineering, LLM được ràng buộc chỉ được phép đọc, tóm tắt và định dạng câu trả lời dựa trên những dữ liệu nền tảng đó.

**Kiểm chứng hiệu quả:** Thông qua nhiều lượt kiểm thử thực nghiệm, kiến trúc RAG này giúp tăng tính xác thực của câu trả lời. Mọi tên phim, năm sản xuất, và điểm số hiển thị trong câu trả lời của Trợ lý ảo đều có nguồn gốc truy xuất (Traceability) trực tiếp từ tập dữ liệu MovieLens 25M, qua đó tăng độ tin cậy đối với người sử dụng dịch vụ.

### 3.2.5 Thách thức 5: Giới hạn Quota API và High Availability

**Mô tả vấn đề:** Để đảm bảo tiêu chí chi phí vận hành bằng 0, hệ thống sử dụng phiên bản API miễn phí do Groq cung cấp [19]. Mặc dù Groq LPU mang lại tốc độ nội suy (Inference speed) rất cao lên tới hàng ngàn token/giây, họ lại thiết lập các giới hạn rất khắt khe về *Số lượng Token mỗi phút (TPM)* và *Số lượng Yêu cầu mỗi phút (RPM)*. Khi lượng người dùng cùng lúc tăng đột biến, hệ thống của Groq sẽ từ chối kết nối, ném ra lỗi ngoại lệ `RateLimitError`, kéo theo nguy cơ gián đoạn toàn bộ hệ thống gợi ý nếu không được xử lý (unhandled exception).

**Cơ chế Smart Fallback:** Khả năng duy trì Tính Sẵn sàng Cao (High Availability) là một yếu tố bắt buộc đối với một sản phẩm hoàn chỉnh. Nhằm bảo vệ luồng tương tác, Tác tử AI được tích hợp cơ chế Dự phòng Thông minh (Smart Fallback). Mã nguồn sử dụng các khối `try-except` bao bọc toàn bộ các lời gọi API. Khi phát hiện hệ thống Groq vượt quá Quota, ứng dụng tự động tạm ngưng bước gọi LLM. Thay vào đó, nó chuyển sang sử dụng mô hình `SentenceTransformer` (chạy nội bộ trên CPU của Hugging Face) để nhúng câu hỏi của người dùng, thực hiện lệnh gọi trực tiếp xuống LanceDB. Mã Python sẽ nhận kết quả trả về, dùng vòng lặp định dạng chuỗi thủ công để tạo ra một danh sách phim ở dạng văn bản (text) có cấu trúc. Người dùng sẽ nhận được thông điệp: *"Trợ lý AI đang bận, dưới đây là các gợi ý dự phòng cho bạn..."* cùng với danh sách kết quả. Mặc dù mức độ tự nhiên trong diễn đạt giảm so với phản hồi của Llama, hệ thống vẫn duy trì khả năng đáp ứng nhu cầu tìm kiếm cốt lõi của người dùng và tránh gián đoạn dịch vụ.

## 3.3 Thách thức phi kỹ thuật

Bên cạnh những rào cản mã nguồn, việc thiết kế hệ thống MLOps còn phải đối mặt với các bài toán quản trị chiến lược, liên quan đến kinh tế học điện toán và hành vi người tiêu dùng.

### 3.3.1 Chi phí hạ tầng và Chuyển dịch mô hình triển khai

Trong giai đoạn thiết kế ban đầu của dự án, kiến trúc hệ thống được định hình theo xu hướng MLOps nguyên thủy (Monolithic MLOps). Kế hoạch dự kiến sử dụng **BentoML** để đóng gói giao diện lập trình ứng dụng (API), sử dụng **Docker Compose** để thiết lập cụm mạng cục bộ (Local Network) kết nối với cơ sở dữ liệu quan hệ PostgreSQL và kho lưu trữ Object Storage nội bộ MinIO. Tuy nhiên, việc đưa khối kiến trúc này lên Internet đòi hỏi phải thuê các máy chủ ảo riêng (VPS) với cấu hình bộ nhớ RAM từ 16GB đến 32GB (chẳng hạn như AWS EC2 hoặc DigitalOcean Droplets). Để đảm bảo ứng dụng luôn trực tuyến phục vụ người dùng 24/7, máy chủ phải hoạt động liên tục (Always-on), tạo ra khoản chi phí duy trì hàng chục đến hàng trăm đô la mỗi tháng. Đây là một rào cản tài chính không thể vượt qua đối với các sinh viên nghiên cứu độc lập.

Quyết định chuyển đổi toàn diện sang kiến trúc **Serverless Cloud-native** là một quyết định thiết kế chiến lược nhằm giải quyết thách thức tài chính này. Nền tảng Hugging Face Spaces cung cấp phần cứng miễn phí cho các ứng dụng có khả năng "tạm dừng" (scale-to-zero) khi không có người dùng truy cập [17]. Cloudflare R2 được lựa chọn làm Data Lake thay thế MinIO và AWS S3 nhờ vào chính sách miễn phí 100% băng thông tải ra (Zero Egress Fees) [18]. Groq cung cấp API miễn phí cho LLM tốc độ cao [19]. Bằng việc phân tán hệ thống lên ba dịch vụ đám mây này, hệ thống đã tối ưu hóa chi phí vận hành (OpEx) về mức xấp xỉ 0.

Quyết định này cho thấy rào cản tài chính có thể được giảm đáng kể khi lựa chọn kiến trúc phần mềm phù hợp cho các ứng dụng AI quy mô lớn.

### 3.3.2 Bài toán Khởi động Nguội (Cold Start Problem)

Thách thức phi kỹ thuật cuối cùng là giải quyết hiện tượng Khởi động Nguội, một vấn đề phổ biến của các hệ thống Lọc Cộng tác. Đối với một người dùng hoàn toàn mới, vừa truy cập vào hệ thống lần đầu tiên, họ chưa từng xem hay chấm điểm bất kỳ bộ phim nào. Ma trận sở thích của họ hoàn toàn trống rỗng, khiến thuật toán Pseudo-Tower Personalization không có trọng số đầu vào để tính toán Vector Trung bình Đại diện.

Cách hệ thống xử lý vấn đề này là thiết lập một luồng chuyển đổi dự phòng (Fallback Routing) dựa trên hành vi người dùng. Khi không có lịch sử người dùng, hệ thống lập tức thay đổi thuật toán xếp hạng, chuyển trọng tâm phân bổ hiển thị dựa trên **Độ phổ biến (Popularity-based Ranking)**. Giao diện người dùng sẽ gợi ý các bộ phim kinh điển (Evergreen) mang tính đại chúng cao nhất (có số lượng `rating_count` lớn và `avg_rating` cao). Cùng với đó, hệ thống cung cấp giao diện hộp chọn đa nhiệm (Multiselect Box) tại thanh bên (Sidebar), khuyến khích người dùng mới chủ động lựa chọn những bộ phim họ biết tên. Ngay khi có từ một đến hai lựa chọn, hồ sơ tương tác của người dùng sẽ được thiết lập, khởi tạo trạng thái cá nhân hóa và kích hoạt luồng tìm kiếm cá nhân hóa thông qua động cơ Vector. Cơ chế điều hướng ổn định này đảm bảo hệ thống luôn có điểm neo để khởi tạo trải nghiệm, bất kể người dùng có lịch sử phong phú hay hoàn toàn vắng dữ liệu.

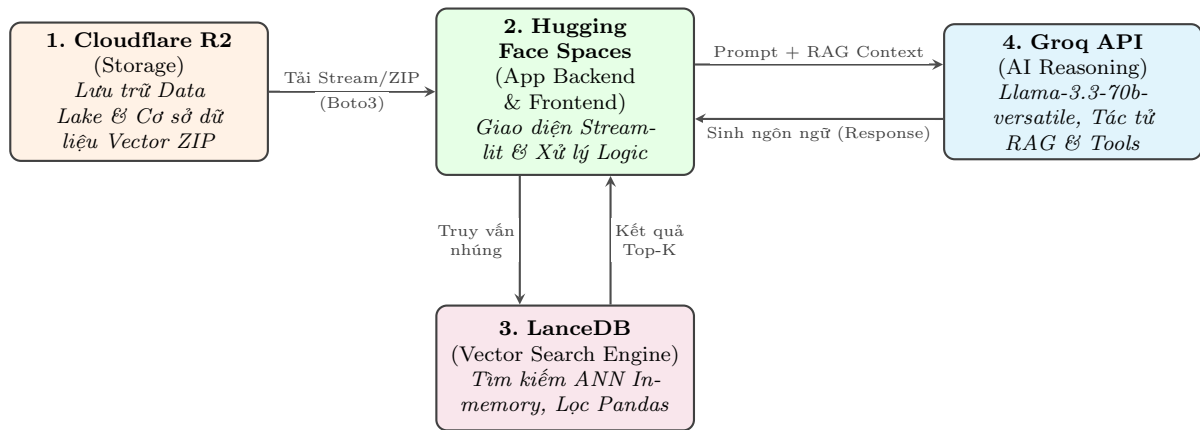
## 4 Kiến trúc Hệ thống và Triển khai Kỹ thuật Hiện tại

Phần này đi sâu vào phân tích kiến trúc kỹ thuật chi tiết của hệ thống gợi ý phim trong phiên bản hiện hành (phiên bản cập nhật năm 2026). Khác với các hệ thống máy học truyền thống phụ thuộc vào hạ tầng máy chủ vật lý công kênh (On-premise) hoặc các phiên bản máy chủ ảo tĩnh tiêu tốn tài nguyên liên tục (Always-on Cloud Instances), hệ thống này được thiết kế và tối ưu hóa dựa trên các nguyên lý điện toán đám mây nguyên bản (Cloud-native) và mô hình kiến trúc phi máy chủ (Serverless). Việc áp dụng các nguyên lý này không chỉ cải thiện khả năng tính toán và xử lý phân tán mà còn giúp giải quyết rào cản lớn nhất của các hệ thống triển khai thao tác học máy (MLOps) trong thực tiễn: chi phí vận hành. Bằng cách thiết kế một đường ống dữ liệu (Data Pipeline) liền mạch, tận dụng các gói dịch vụ miễn phí và công nghệ mã nguồn mở, hệ thống đạt được độ trễ truy vấn thấp, trong khi chi phí duy trì hệ thống trong thời gian chờ (idle time) được đưa về mức tiệm cận không [17, 18, 16, 19].

### 4.1 Tổng quan kiến trúc

Kiến trúc tổng thể của hệ thống được tổ chức thành một đường ống xử lý dữ liệu một chiều (Unidirectional Pipeline), đảm bảo tính module hóa cao và sự tách biệt trách nhiệm (Separation of Concerns) giữa các thành phần cốt lõi. Sự phân tách này cho phép hệ thống dễ dàng mở rộng theo chiều ngang (scale out), thay thế hoặc nâng cấp từng thành phần phần mềm mà không ảnh hưởng dây chuyền đến các phân hệ khác. Đường ống luân chuyển thông tin được cấu thành từ bốn lớp chính: **Cloudflare R2 (Lớp Lưu trữ)** → **Hugging Face Spaces (Lớp Ứng dụng Frontend và Backend)** → **LanceDB (Lớp Tìm kiếm Vector)** → **Groq API (Lớp Suy luận Trí tuệ Nhân tạo)**.

Dưới đây là sơ đồ kiến trúc biểu diễn trực quan các thành phần cấu trúc và sự tương tác giữa chúng trong không gian đám mây:



**Hình 1:** Sơ đồ luồng dữ liệu và kiến trúc tương tác giữa các dịch vụ Cloud-native

Lớp đầu tiên là hệ thống lưu trữ đối tượng phân tán (Object Storage) do Cloudflare R2 đảm nhiệm, đóng vai trò là một Hồ dữ liệu (Data Lake) trung tâm chứa khối lượng dữ liệu thô và các khối dữ liệu Vector đã được tiền xử lý. Dữ liệu từ đây sẽ được luân chuyển (streaming) qua mạng Internet một cách an toàn để đi vào Lớp Ứng dụng đặt tại nền tảng Hugging Face Spaces. Tại máy chủ của Hugging Face, nền tảng Streamlit sẽ chịu trách nhiệm hiển thị giao diện người dùng (Frontend), đồng thời đóng vai trò làm Backend để điều phối các truy vấn logic phức tạp. Khi có yêu cầu liên quan đến tìm kiếm dữ liệu không gian nhiều chiều, Hugging Face sẽ chuyển tiếp lệnh cho lõi LanceDB hoạt động trực tiếp trong bộ nhớ (In-memory) để truy xuất các bộ phim tương đồng. Cuối cùng, kết quả truy xuất thô từ cơ sở dữ liệu sẽ được đưa vào Tác tử AI của Groq, cho phép mô hình ngôn ngữ Llama biến đổi dữ liệu thành các phản hồi tự nhiên hơn và giải thích rõ ràng lý do cho người dùng.

Tổng kết lại cho phần kiến trúc tổng quan, thông qua sự tích hợp nhịp nhàng và phân rã các lớp này, hệ thống đã giảm đáng kể sự phụ thuộc vào các rào cản vật lý cố hữu. Sự luân chuyển dữ liệu từ lưu trữ tĩnh (Storage) qua bộ xử lý động (App Server) đến các bộ vi xử lý nơ-ron chuyên dụng (LPU của Groq) được thực thi trong thời gian chỉ tính bằng phần nghìn giây, tạo ra một hệ thống liền mạch, tốc độ cao và mở ra bước đệm phù hợp để đi sâu vào phân tích luồng dữ liệu đầu vào.

## 4.2 Đường ống Tiền xử lý Dữ liệu (Data Pipeline)

Quá trình chuẩn bị, dọn dẹp và làm giàu dữ liệu là khâu nền tảng quyết định chất lượng của bất kỳ hệ thống máy học nào. Toàn bộ kịch bản tiền xử lý dữ liệu của hệ thống được tập trung và thực thi trong một tệp tin Jupyter duy nhất là `notebooks/colab_pipeline.ipynb`. Việc sử dụng môi trường tính toán đám mây Google Colab cung cấp quyền truy cập tạm thời (ephemeral) vào tài nguyên bộ nhớ RAM và năng lực CPU/GPU mạnh, đặc biệt phù hợp cho các tác vụ biến đổi dữ liệu (ETL) tốn tài nguyên trên khối dữ liệu lớn.

### 4.2.1 Dòng dữ liệu và thống kê sau từng bước trong notebook

Notebook `colab_pipeline.ipynb` hiện thực hóa pipeline theo hướng data lineage rõ ràng: dữ liệu thô được tải từ Cloudflare R2, xử lý trên Google Colab bằng PySpark/Pandas, làm giàu bằng TMDb API, mã hóa bằng SentenceTransformer, ghi vào LanceDB, log thí nghiệm lên DagsHub/MLflow, rồi nén artifact `lancedb_movies.zip` và đẩy ngược về R2 để ứng dụng Hugging Face tải về khi phục vụ người dùng. Bảng 1 tóm tắt các bước chính và kết quả trung gian quan sát được trong notebook.

**Bảng 1:** Tóm tắt các bước xử lý trong notebook Colab pipeline

| No. | Thành phần         | Mục đích xử lý                                                                                                                                                                    | Kết quả/ghi chú                              |
|-----|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| 1   | R2 + boto3         | Tải các tệp <code>movies.csv</code> , <code>links.csv</code> , <code>tags.csv</code> , <code>ratings.csv</code> , <code>genome-scores.csv</code> , <code>genome-tags.csv</code> . | Tạo bản sao dữ liệu thô trong Colab.         |
| 2   | PySpark            | Gom nhóm <code>ratings.csv</code> , tính <code>avg_rating</code> , <code>rating_count</code> , lọc phim có ít nhất 50 lượt đánh giá.                                              | 13,176 quality movies.                       |
| 3   | Pandas + Genome    | Lọc genome score có <code>relevance</code> $\geq 0.8$ , lấy tối đa 10 genome tags/phim.                                                                                           | 13,498 phim có genome tags.                  |
| 4   | Pandas + User Tags | Chuẩn hóa tag cộng đồng, đếm tần suất và lấy tối đa 5 user tags/phim.                                                                                                             | 45,251 phim có user tags.                    |
| 5   | Master Merge       | Merge quality movies, metadata phim, liên kết TMDB, genome tags và user tags.                                                                                                     | 13,164 phim trong master table.              |
| 6   | TMDB API           | Fetch <code>overview</code> và <code>poster_path</code> ; dùng retry, timeout và checkpoint để chịu lỗi.                                                                          | Khoảng 13,160 phim cần bổ sung TMDB.         |
| 7   | Text Enhancement   | Tạo <code>combined_text</code> từ title, genres, overview, user tags, genome tags và quality signal.                                                                              | Chuỗi đầu vào giàu ngữ cảnh cho embedding.   |
| 8   | ST + LanceDB       | Mã hóa bằng <code>all-MiniLM-L6-v2</code> , batch size 64, ghi vector 384 chiều vào LanceDB.                                                                                      | Bảng <code>movies</code> với chỉ mục IVF-PQ. |
| 9   | MLflow + R2        | Log tham số/metric lên DagsHub MLflow, nén và upload <code>lancedb_movies.zip</code> .                                                                                            | Artifact phục vụ ứng dụng production.        |

*Ghi chú đối chiếu số liệu:* các mốc 13,176 và 13,164 trong Bảng 1 là output trực tiếp của notebook sau bước lọc chất lượng và merge TMDB. Trong ảnh dashboard ở Hình 10, ô “Số phim Vectorized” hiển thị 27,278; con số này thuộc lớp hiển thị/artifact phục vụ của ứng dụng và được giữ riêng khi mô tả ảnh dashboard, tránh nhầm với bảng master sau join trong notebook.

#### 4.2.2 Tải dữ liệu và tinh lọc bản ghi

Quá trình khởi chạy của đường ống bắt đầu bằng việc hệ thống tự động tải xuống tập dữ liệu chuẩn *MovieLens 25M* từ kho lưu trữ Cloudflare R2 thông qua giao thức S3 (sử dụng thư viện `boto3`). Tập dữ liệu thô này chứa hơn 25 triệu bản ghi tương tác, bao gồm thông tin chi tiết về phim (`movies.csv`), lịch sử đánh giá (`ratings.csv`), các nhãn dán thủ công của cộng đồng (`tags.csv`), và cấu trúc liên kết gen biểu diễn mức độ tương quan ngữ nghĩa (`genome-scores.csv`). Đối mặt với đặc tính nhiều dữ liệu tự nhiên và độ thưa thớt cao (sparsity) của ma trận tương tác, bước kỹ thuật đầu tiên và quan trọng nhất là thiết lập một bộ cổng lọc kiểm soát chất lượng (Quality Gate).

Hệ thống tiến hành nhóm các bản ghi (group by) dựa trên mã định danh phim và kiên quyết loại bỏ toàn bộ các bộ phim có tổng số lượt đánh giá nhỏ hơn 50. Theo nghiên cứu thực nghiệm, các bộ phim không vượt qua được ngưỡng cơ sở này thường là các tác phẩm ít phổ biến, các bộ phim thiếu dữ liệu xác thực thiếu tính xác thực và có khả năng làm sai lệch (bias) các dự đoán của mạng nơ-ron học sâu do thiếu hụt thông tin tương tác bề mặt. Sau khi quá trình lọc nghiêm ngặt này hoàn tất, số lượng phim hợp lệ được theo dõi theo từng mốc pipeline: notebook ghi nhận 13,176 phim sau lọc rating và 13,164 phim trong bảng master sau khi merge liên kết TMDB, còn dashboard phục vụ hiển thị 27,278 phim ở lớp thống kê vectorized. Tiếp theo, hệ thống thực hiện hàng loạt các phép kết nối bảng (Table Joins) qua thư viện Pandas, hợp nhất dữ liệu từ nhiều tệp rời rạc khác nhau thành một Bảng Phẳng (Flat Table) duy nhất, chứa toàn bộ đặc trưng cần thiết của một thực thể phim.

#### 4.2.3 Kỹ thuật Tăng cường Văn bản Nhúng (Text Embedding Enhancement)

Để quá trình nhúng vector (Vector Embedding) đạt được độ phân giải ngữ nghĩa cao nhất có thể, hệ thống không chỉ áp dụng kỹ thuật ghép nối tiêu đề và mô tả cơ bản. Thay vào đó, dự



án đã đề xuất và phát triển một phương pháp chuyên sâu mang tên **Tăng cường Văn bản Nhúng (Text Embedding Enhancement)**.

Trong phương pháp này, mỗi dòng dữ liệu của bộ phim được tái cấu trúc thành một *Chuỗi Văn bản Làm giàu (Enriched Text)*. Đặc biệt, bên cạnh các thông tin về thể loại và từ khóa, hệ thống chủ động chèn thêm một mã thông báo ảo (virtual token) biểu diễn một cách rõ ràng cho chất lượng và độ uy tín thực tế của phim. Chuỗi văn bản đầu vào được chuẩn hóa sẽ có dạng: "[Title] *The Matrix*. [Genres] *Action/Sci-Fi*. [Overview] *A computer hacker learns from mysterious rebels about the true nature of his reality*. [User Tags] *cyberpunk, philosophical*. [Quality] *Rating: 4.8/5.0, 50000 votes*".

Việc chèn rõ ràng cụm thông tin [Quality] đóng vai trò quan trọng trong không gian hình học. Thao tác này giúp cơ chế chú ý (Attention Mechanism) của mô hình học sâu **SentenceTransformer** (cụ thể là kiến trúc *all-MiniLM-L6-v2*) liên kết chất lượng điện ảnh với ngữ nghĩa của cốt truyện [10]. Kết quả là trong không gian vector 384 chiều, các bộ phim có tín hiệu đánh giá cao có xu hướng tập hợp thành các cụm (clusters) tách biệt hơn so với các bộ phim chất lượng thấp, dù chúng có thể có chung một vài từ khóa mô tả về cốt truyện.

#### 4.2.4 Xây dựng Không gian LanceDB và Đánh giá MLOps

Sau khi quá trình mã hóa (encoding) sinh ra khoảng 13.000 vector chuẩn hóa, hệ thống tiến hành nạp toàn bộ lượng dữ liệu này vào cơ sở dữ liệu LanceDB. Để khắc phục triệt để các lỗi về hình thái dữ liệu (tensor shape errors) thường xuyên phát sinh khi Python tương tác với lõi biên dịch bằng Rust của LanceDB, lược đồ cơ sở dữ liệu (Schema) được khai báo một cách khắt khe thông qua thư viện **pyarrow**. Thay vì lưu trữ vector dưới dạng danh sách động không xác định (variable-length list), trường **vector** được thiết lập chặt chẽ với kiểu **FixedSizeList(384)**.

Cùng với quá trình tạo lập bảng, cấu trúc chỉ mục lượng tử hóa **IVF-PQ** (Inverted File Index kết hợp Product Quantization) có thể được định nghĩa hoặc bỏ qua để thay bằng cơ chế Tìm kiếm Tuần tự (Flat Search) tùy theo cấu hình thử nghiệm. Mặc dù tập dữ liệu hiện tại chỉ có kích thước nhỏ, việc chuẩn bị sẵn cấu trúc **FixedSizeList** giúp hệ thống sẵn sàng mở rộng lên hàng chục triệu vector trong tương lai mà vẫn duy trì tốc độ truy xuất nhanh (sub-millisecond). Xuyên suốt quá trình xây dựng đường ống, các số liệu đo lường MLOps quan trọng như kích thước tệp, độ dài trung bình của nhãn dán, cũng như các chỉ số hồi quy xấp xỉ giả lập như *Recall@10* và *NDCG@10* được lưu vết tự động lên máy chủ **DagsHub/MLflow**. Cuối cùng, thư mục chứa cơ sở dữ liệu **lancedb\_movies** được đóng gói thành tệp nén **.zip** và tải ngược lên Cloudflare R2 Data Lake, kết thúc một vòng đời xử lý hoàn chỉnh.

#### 4.2.5 Schema LanceDB và tham số chỉ mục

Từ notebook, bảng LanceDB **movies** không được tạo theo kiểu động mà được khai báo schema tường minh bằng **pyarrow**. Cách làm này quan trọng vì LanceDB dựa trên Apache Arrow; nếu vector bị ghi dưới dạng danh sách độ dài biến thiên, truy vấn vector search có thể phát sinh lỗi hình dạng dữ liệu hoặc giảm hiệu năng. Bảng 2 trình bày schema phục vụ truy xuất.

Chỉ mục vector được tạo theo metric **cosine** với cấu hình IVF-PQ gồm **num\_partitions=256** và **num\_sub\_vectors=16** [16]. Với tập khoảng 13 nghìn phim, cấu hình này không chỉ đủ nhanh cho truy vấn hiện tại mà còn đóng vai trò mẫu thiết kế để mở rộng lên tập vector lớn hơn. Notebook cũng có nhánh fallback cho khác biệt API giữa các phiên bản LanceDB: nếu tham số **vector\_column\_name** không được hỗ trợ, mã nguồn chuyển sang tham số **column**. Chi tiết này nên được xem là một quyết định MLOps thực tế nhằm giảm rủi ro hỏng pipeline khi thư viện thay đổi phiên bản.

Nhìn lại toàn bộ Đường ống Tiền xử lý, có thể khẳng định rằng tầng hệ thống này hoạt

**Bảng 2:** Schema của bảng movies trong LanceDB

| Trường       | Kiểu dữ liệu                 | Vai trò trong hệ thống                                                    |
|--------------|------------------------------|---------------------------------------------------------------------------|
| movieId      | int32                        | Khóa định danh phim từ MovieLens, dùng để lookup và mapping nội bộ.       |
| title        | string                       | Tên phim hiển thị cho người dùng và chatbot.                              |
| genres       | string                       | Thể loại phim, vừa phục vụ lọc metadata vừa bổ sung ngữ nghĩa cho prompt. |
| overview     | string                       | Tóm tắt nội dung lấy từ TMDB, là nguồn ngữ nghĩa chính cho embedding.     |
| poster_path  | string                       | Đường dẫn poster để giao diện có thể hiển thị trực quan kết quả.          |
| avg_rating   | float32                      | Điểm đánh giá trung bình, dùng trong quality signal và reranking.         |
| rating_count | int32                        | Số lượt đánh giá, dùng để lọc độ tin cậy và tính popularity.              |
| vector       | FixedSizeList (float32, 384) | Vector embedding 384 chiều do all-MiniLM-L6-v2 sinh ra.                   |

động như một lớp chuẩn hóa và làm giàu thông tin. Bằng việc phân tách rõ ràng trách nhiệm tiền xử lý sang môi trường Colab, hệ thống đã giảm tải đáng kể khối lượng tính toán quy mô lớn cho các tầng ứng dụng phục vụ trực tiếp người dùng ở giai đoạn sau.

#### 4.2.6 Tính tái lập và quản lý bí mật thực nghiệm

Để chạy lại pipeline, notebook yêu cầu một tập biến bí mật được cấu hình trong Google Colab Secrets thay vì ghi cứng trực tiếp vào mã nguồn. Các nhóm biến cần thiết gồm:

- **Cloudflare R2:** R2\_ACCESS\_KEY, R2\_SECRET\_KEY, R2\_ENDPOINT.
- **Làm giàu dữ liệu:** TMDB\_API\_KEY.
- **Tracking:** MLFLOW\_TRACKING\_URI, DAGSHUB\_USERNAME, DAGSHUB\_TOKEN.

Cách tổ chức này giúp pipeline tái lập được trên Colab nhưng vẫn tránh rò rỉ khóa truy cập trong báo cáo hoặc repository.

Một điểm quan trọng về mặt đánh giá là notebook hiển log Recall\_k10 và NDCG\_k10 lên MLflow bằng giá trị mô phỏng để kiểm tra luồng tracking. Do đó, các metric này phù hợp để chứng minh khả năng MLOps và logging, nhưng chưa nên diễn giải như kết quả benchmark offline cuối cùng nếu chưa bổ sung tập kiểm thử lịch sử người dùng và quy trình tính metric xác định. Trong các phiên bản tiếp theo, pipeline có thể thay phần mô phỏng này bằng holdout theo thời gian hoặc leave-one-out trên lịch sử rating để biến MLflow thành nguồn đánh giá định lượng chính thức.

### 4.3 Động cơ Tìm kiếm Vector (Vector Search Engine)

Động cơ tìm kiếm Vector là thành phần lõi điều phối luồng truy vấn của toàn bộ lớp Backend, được định nghĩa một cách rành mạch trong tệp mã nguồn `src/serving/semantic_search.py`. Nhiệm vụ quan trọng nhất của lớp này là kết nối ứng dụng với khối dữ liệu không gian đa chiều, thực thi các phép nội suy khoảng cách Euclid hay Cosine một cách tối ưu, qua đó trả về danh sách các ứng viên (candidates) có độ tương đồng ngữ nghĩa cao nhất.

### 4.3.1 Khởi tạo và Tối ưu Lọc Siêu dữ liệu (Pandas Filtering)

Khi vùng chứa (Container) của ứng dụng Hugging Face Spaces bắt đầu quy trình khởi động cục bộ, hệ thống sẽ tự động gọi thư viện boto3 để kéo tệp tin `lancedb_movies.zip` từ bộ nhớ đám mây Cloudflare R2 về ổ đĩa cứng ảo. Dữ liệu được giải nén nhanh chóng để LanceDB khởi tạo kết nối. Lợi điểm kiến trúc lớn nhất của một cơ sở dữ liệu dạng nhúng (embedded database) là nó chạy trực tiếp trên vùng nhớ RAM hiện tại và luồng thực thi chung với tiến trình ứng dụng Python. Cơ chế này loại bỏ hoàn toàn độ trễ giao tiếp mạng (Network I/O) vốn là nút thắt cổ chai khi giao tiếp với các máy chủ cơ sở dữ liệu độc lập bên ngoài.

Trong quá trình thực thi và kiểm thử thực tế, hệ thống đã phát hiện các sự cố liên quan đến động cơ DataFusion (engine lõi phân tích SQL tích hợp bên trong LanceDB). Các truy vấn metadata tĩnh (như tìm kiếm theo định danh chính xác `movieId`) thường xuyên ném ra các lỗi ngoại lệ (exceptions) do công cụ này tự động chuẩn hóa sai lệch kiểu chữ cái (lowercase normalization) đối với lược đồ (schema) của Arrow. Giải pháp thiết kế được áp dụng để thay thế là **Lọc bằng Pandas (Pandas Metadata Filtering)**. Vì dữ liệu sau khi lọc (chỉ khoảng 13.000 bộ phim) có kích thước lưu trữ nhỏ gọn, hệ thống chuyển toàn bộ bảng cơ sở dữ liệu thành định dạng DataFrame của Pandas trực tiếp trong RAM. Khi có yêu cầu tìm kiếm chính xác, mọi thao tác lọc (filtering) đều được thực hiện bằng thư viện Pandas. Sự kết hợp song song này tận dụng Vector Search của LanceDB cho không gian nhiều chiều và Pandas cho metadata một chiều, đảm bảo độ chính xác cao mà không gây phát sinh lỗi cú pháp.

### 4.3.2 Cá nhân hóa Giả lập (Pseudo-Tower Personalization)

Một trong những thách thức kỹ thuật lớn nhất đối với các ứng dụng triển khai theo hướng Serverless là việc hệ thống không có đủ tài nguyên GPU hoặc thời gian chạy để liên tục huấn luyện các mô hình lọc cộng tác (Collaborative Filtering) thông thường. Nhằm mang lại khả năng cá nhân hóa kết quả chuyên sâu mà không cần đến kiến trúc mô hình Mạng nơ-ron hai tháp (Two-Tower Neural Networks) phức tạp, đề tài đề xuất kiến trúc **Pseudo-Tower Personalization** (Cá nhân hóa giả lập tháp đôi).

Thuật toán trích xuất sở thích người dùng bắt đầu bằng cách yêu cầu người dùng cung cấp một danh sách các bộ phim mà họ đã xem và đánh giá cao. Dựa trên danh sách đó, mã nguồn hệ thống lập qua các vector phim trong cơ sở dữ liệu nội bộ và tính toán **Trung bình có trọng số (Weighted Average Vector)**. Điểm số đánh giá của người dùng được sử dụng trực tiếp làm trọng số toán học (weight) kéo khối tâm của mảng vector về phía mình. Kết quả trả về là một *Vector đại diện (User Profile Vector)* phản ánh chính xác phương hướng sở thích của người dùng trong không gian 384 chiều, phục vụ trực tiếp cho việc gọi hàm tìm kiếm xấp xỉ K-láng giềng gần nhất (ANN KNN) để tìm ra các bộ phim nằm lân cận khối tâm đó.

### 4.3.3 Tinh chỉnh xếp hạng (Reranker Layer)

Bên cạnh khả năng tìm kiếm cá nhân hóa, một đặc thù khó tránh khỏi của thuật toán đo khoảng cách Cosine (Cosine Similarity) là xu hướng trả về những bộ phim quá ngách (niche) hoặc có nội dung trùng lặp cao. Để cân bằng hệ sinh thái nội dung, một **Lớp Tinh chỉnh Xếp hạng (Reranker Layer)** đã được thiết kế và bổ sung. Danh sách ứng viên sơ cấp (Top-K) do LanceDB trả về sẽ được tính điểm lại một lần nữa dựa trên phương trình hồi quy lại:

$$Final\_Score = 0.6 \times Similarity\_Score + 0.3 \times Popularity\_Score + 0.1 \times Quality\_Score$$

Hệ thống phân bổ ba trọng số 60% – 30% – 10% nhằm đảm bảo tính cá nhân hóa (độ tương đồng) tiếp tục đóng vai trò chủ đạo trong định hướng nội dung. Tuy nhiên, nó cũng ưu

tiên hiển thị các bộ phim mang tính đại chúng (Popularity) và được đánh giá cao (Quality), qua đó hạn chế tình trạng gợi ý những bộ phim có chất lượng thấp chỉ vì khớp từ khóa.

Dúc kết lại cho phần Động cơ Tìm kiếm Vector, thông qua việc kết hợp Pandas và LanceDB, hệ thống mô phỏng được một số đặc tính của hệ thống Lọc Cộng tác quy mô lớn mà không cần duy trì hạ tầng học sâu riêng biệt.

#### 4.4 Tác tử Trí tuệ Nhân tạo (LLM Agent)

Tầng thứ ba của kiến trúc là lớp xử lý ngôn ngữ tự nhiên, được xây dựng và đóng gói trong tệp `src/serving/chatbot.py`. Sự tích hợp công nghệ trí tuệ nhân tạo sinh tạo (Generative AI) nâng cấp hệ thống gợi ý từ việc trình bày danh sách thông thường thành một luồng giao tiếp tương tác hai chiều có khả năng giải thích, nơi người dùng có thể tranh luận và tìm hiểu lý do đằng sau mỗi gợi ý. Mô hình nền tảng được chọn để đảm nhiệm vai trò này là **Llama-3.3-70b-versatile**, được triển khai thông qua máy chủ hiệu năng cao của **Groq API**.

##### 4.4.1 Mô hình và Giao thức Gọi hàm (Function Calling)

Khác với các ứng dụng trò chuyện thông thường, hệ thống này không chỉ gửi tin nhắn văn bản đến LLM để nhận về câu trả lời sinh tự do. Llama 3.3 được thiết lập để hoạt động như một **Tác tử (AI Agent)** có khả năng nhận diện ý định và kích hoạt các công cụ phân tích dữ liệu chuyên sâu (Tools). 5 công cụ cụ thể đã được lập trình dưới dạng lược đồ cấu trúc OpenAI-compatible [19] bao gồm:

1. Khởi tạo tìm kiếm phim theo ngữ nghĩa và mô tả nội dung văn bản.
2. Truy xuất danh sách phim tương tự dựa trên một định danh phim có sẵn.
3. Truy xuất các bộ phim được đánh giá cao dựa trên mốc thời gian thập kỷ (ví dụ: thập niên 90).
4. Phân tích và so sánh định lượng, định tính giữa hai bộ phim riêng biệt.
5. Lọc và tìm kiếm các bộ phim xu hướng dựa trên ngưỡng điểm chất lượng nhất định.

Sự kết hợp giữa năng lực suy luận của hệ thống 70 tỷ tham số và giao thức Gọi hàm cho phép LLM chủ động phân tích các câu hỏi phức tạp thành các câu lệnh truy vấn hàm cụ thể, tạo ra tính linh hoạt cao hơn so với các truy vấn tĩnh (static query).

##### 4.4.2 Kiến trúc RAG nguyên bản (Retrieval-Augmented Generation)

Bên cạnh khả năng giao tiếp, sự kết hợp giữa Function Calling và mẫu thiết kế **RAG nguyên bản** (Retrieval-Augmented Generation) là điểm quan trọng của Tác tử AI này [12]. Khi tiếp nhận truy vấn tự do từ người dùng, hệ thống phân tích và gọi hàm tìm kiếm tương ứng xuống động cơ LanceDB (bước *Retrieval*). Danh sách phim thực tế được trả về, sau đó được định dạng thành chuỗi ngữ cảnh chặt chẽ (Context) và bổ sung trực tiếp vào Chỉ thị Hệ thống (System Prompt).

Lúc này, Llama đóng vai trò định dạng ngôn ngữ, giải thích logic, và tóm tắt cốt truyện (bước *Generation*). Mô hình được yêu cầu không sử dụng bộ nhớ tham số (parametric memory) để tự tạo thông tin ngoài ngữ cảnh truy xuất. Quy trình thiết kế này giúp hạn chế hiện tượng Ảo giác AI (Hallucination), giảm rủi ro mô hình tự động sinh ra tên các bộ phim không có thật hoặc nhầm lẫn thông tin đạo diễn.

### 4.4.3 Bộ nhớ Thực thể và Chế độ Dự phòng thông minh

Trong bối cảnh của các cuộc đàm thoại kéo dài (Multi-turn conversations), việc duy trì luồng ngữ cảnh cá nhân hóa là rất quan trọng để tạo cảm giác tự nhiên. Biến `entity_memory` được cấu trúc tại lớp ứng dụng để ghi nhận các từ khóa thể loại mà người dùng ưa thích (chẳng hạn như "hành động", "khoa học viễn tưởng", "kinh dị"). Tập hợp các sở thích ẩn này được tự động chèn vào tham số ẩn của Tác tử, giúp LLM "ghi nhớ" xu hướng của người dùng mà không cần người dùng phải lặp lại thông tin đó trong các câu truy vấn tiếp theo.

Mặt khác, đối mặt với các rủi ro về giới hạn truy cập API (Quota Limit) của hệ thống Groq cấp độ miễn phí, ứng dụng được trang bị cơ chế **Dự phòng Thông minh (Smart Fallback)**. Trong trường hợp xảy ra lỗi quá tải `RateLimitError`, hệ thống sẽ tự động bỏ qua bước gọi LLM. Thay vào đó, mã nguồn Python sẽ trực tiếp nhúng truy vấn bằng mô hình `SentenceTransformer` (chạy nội bộ), gọi hàm tìm kiếm LanceDB và định dạng chuỗi văn bản tĩnh gửi đến người dùng kèm thông điệp báo hiệu chế độ dự phòng. Cơ chế này cải thiện Tính sẵn sàng cao (High Availability), giúp người dùng vẫn nhận được gợi ý nội dung khi hạ tầng bên thứ ba gặp sự cố hoặc bị giới hạn hạn mức.

Khép lại cấu trúc của Tác tử Trí tuệ Nhân tạo, sự tách bạch rõ ràng giữa khả năng truy xuất thông tin của nền tảng RAG và khả năng suy luận ngôn ngữ của LLM giúp tăng tính minh bạch. Cách tổ chức này hỗ trợ các nhà phát triển và kỹ sư dễ dàng gỡ lỗi, kiểm soát, và duy trì chất lượng của mọi đề xuất trước khi chúng tới người dùng.

## 4.5 Giao diện Người dùng và Phân tích Trực quan (Streamlit)

Lớp trên cùng của hệ thống kiến trúc là không gian tương tác người dùng, được thiết kế và quản lý toàn diện bởi mã nguồn trong tệp `app.py`. Nền tảng framework mã nguồn mở **Streamlit** được sử dụng nhằm mục đích triển khai giao diện người dùng theo phong cách ứng dụng web lai (Thin Client). Ứng dụng cung cấp hai trải nghiệm hoàn toàn riêng biệt trên cùng một thanh điều hướng, phục vụ cả nhu cầu phân tích chuyên sâu của quản trị viên lẫn nhu cầu giải trí của người dùng phổ thông.

### 4.5.1 Bảng điều khiển Phân tích (Analytics Dashboard)

Bảng điều khiển dữ liệu (Dashboard) được phân vùng thành 5 Không gian làm việc (Tabs) độc lập: *Tổng quan hệ thống*, *Chất lượng Phim*, *Hành vi Người dùng*, *Phân tích Tag & Genome*, và *MLOps Metrics*. Để phục vụ khả năng trực quan hóa đồ thị đa chiều phức tạp một cách thẩm mỹ, hệ thống tích hợp sâu với thư viện đồ họa **Plotly Express**.

Thách thức kỹ thuật lớn nhất đối với việc vẽ biểu đồ trực tuyến trên môi trường đám mây miễn phí là lỗi tràn bộ nhớ (Out-Of-Memory). Để khắc phục vấn đề này, module dữ liệu không tải toàn bộ 25 triệu dòng dữ liệu vào máy ảo. Nó duy trì kết nối truyền phát tới Cloudflare R2 để tải luồng dữ liệu `ratings.csv` và `movies.csv` với giới hạn tối đa 500.000 dòng. Tập dữ liệu mẫu đại diện này cung cấp các thuộc tính phân phối thống kê cần thiết để hệ thống phác họa các đồ thị như Bản đồ nhiệt mật độ xem phim theo giờ (Density Heatmap), Biểu đồ cây phân loại thể loại (Treemap), và Phân tích độ lệch chuẩn của đánh giá (Rating Bias). Đây là một thiết kế giúp đảm bảo sự ổn định của ứng dụng web trong khi vẫn giữ được giá trị báo cáo dữ liệu.

### 4.5.2 Tối ưu hóa Trải nghiệm Người dùng (UX)

Về phía dịch vụ gợi ý cá nhân hóa và tương tác hội thoại, hệ thống tập trung vào Trải nghiệm Người dùng (User Experience). Việc buộc người dùng phải ghi nhớ mã định danh bằng số

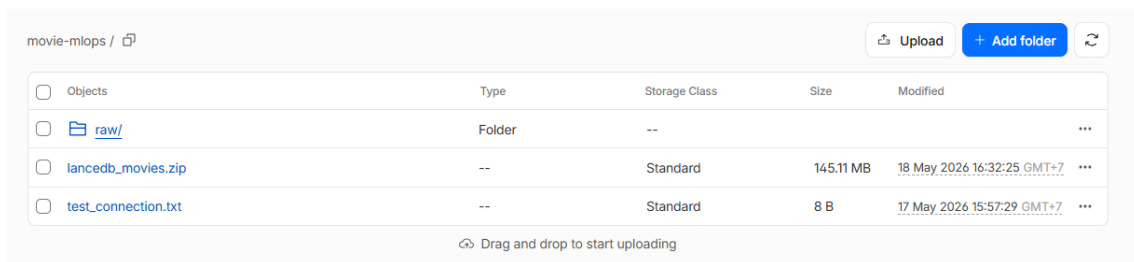


(Movie ID) của từng bộ phim để khởi tạo luồng Pseudo-Tower là một điểm yếu quan trọng của các hệ thống nguyên mẫu cũ. Hệ thống hiện hành đã cải thiện hạn chế này thông qua việc xây dựng một bộ Từ điển ánh xạ thời gian thực (Lookup Dictionary).

Bằng cách quét tự động cơ sở dữ liệu LanceDB một lần duy nhất lúc khởi động để tạo từ điển liên kết giữa *Tên Phim (Title)* và *Mã ID*, người dùng giờ đây có thể thao tác với các danh sách chọn lọc đa nhiệm (Multiselect Box) bằng tên tự nhiên của các tác phẩm điện ảnh. Bất cứ khi nào danh sách được gửi đi, mã nguồn Python sẽ tự động dịch ngược (reverse map) từ tên sang mã số ID hệ thống để tương tác với động cơ không gian Vector bên dưới. Sự điều chỉnh thiết kế này giúp một ứng dụng học thuật có tính kỹ thuật cao trở nên dễ tiếp cận hơn với người dùng cuối.

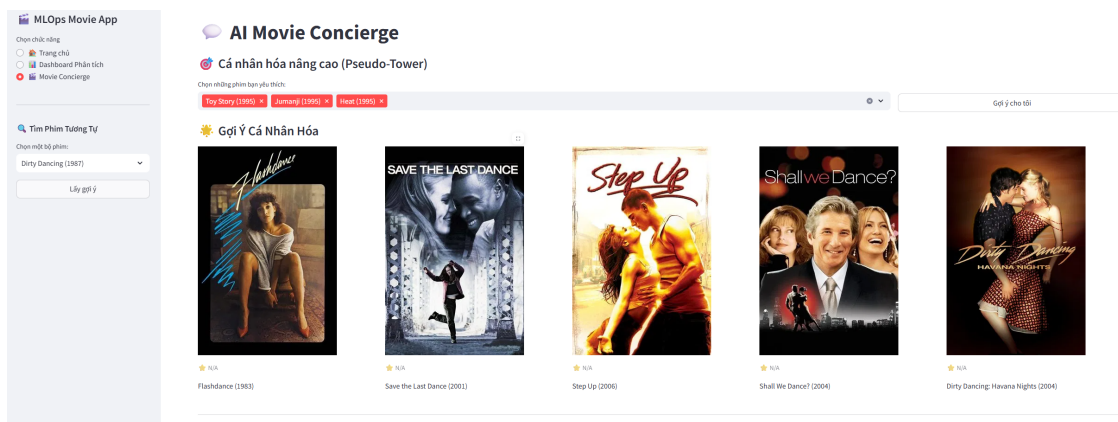
#### 4.5.3 Minh họa triển khai và trải nghiệm người dùng

Các hình ảnh chụp từ phiên bản ứng dụng hiện tại cho thấy pipeline không chỉ dừng ở mức mô hình thử nghiệm trong notebook, mà đã được đóng gói thành một quy trình triển khai có thể quan sát và kiểm chứng. Ở tầng lưu trữ, Cloudflare R2 đóng vai trò kho artifact trung tâm, nơi lưu các tệp dữ liệu đã tiền xử lý và cơ sở dữ liệu vector đã nén. Thiết kế này giúp tách biệt rõ ràng giữa quá trình huấn luyện/chuẩn bị dữ liệu trong notebook và quá trình phục vụ truy vấn ở ứng dụng Streamlit.



**Hình 2:** Minh họa lớp lưu trữ artifact trên Cloudflare R2 dùng cho dữ liệu và cơ sở vector

Ở tầng tương tác, giao diện gợi ý được thiết kế để người dùng có thể nhập một hoặc nhiều bộ phim yêu thích bằng tên tự nhiên thay vì mã định danh nội bộ. Điều này trực tiếp giải quyết điểm nghẽn UX của các prototype học thuật: người dùng không cần biết cấu trúc dữ liệu phía sau, trong khi backend vẫn có thể ánh xạ tên phim sang vector tương ứng để tính hồ sơ sở thích.



**Hình 3:** Giao diện gợi ý dựa trên một hoặc nhiều phim đầu vào do người dùng lựa chọn

Ngoài chế độ gợi ý trực tiếp, hệ thống còn cung cấp giao diện hội thoại để người dùng đặt câu hỏi tự nhiên về sở thích, thể loại, hoặc danh sách phim mong muốn. Ảnh chụp giao diện

chatbot minh họa vai trò của lớp LLM/RAG: mô hình ngôn ngữ không thay thế cơ sở dữ liệu, mà đóng vai trò diễn giải kết quả truy xuất, trình bày lý do và duy trì mạch hội thoại thân thiện hơn với người dùng cuối.

#### Chat với AI (RAG & Entity Memory Active)

🔍 tìm phim thập niên 90

📁 Dựa vào thông tin database, tôi có thể tìm thấy phim "Underground" (1995) thuộc thập niên 90.

- Phim: Underground (1995) | Thể loại: Comedy/Drama/War | Điểm: 3.950000/47683716 ⭐ Nội dung: Black marketeers Marko and Blacky manufacture and sell weapons to the Communist resistance in WWII Belgrade, living the good life along the way. Marko's surreal duplicity propels him up the ranks of the Communist Party, and he eventually abandons Blacky and steals his girlfriend. After a lengthy stay in a below-ground shelter, the couple reemerges during the Yugoslavian Civil War of the 1990s as Marko sees the opportunity to exploit the situation.

🔍 Bạn gợi ý cho tôi xem các bộ phim lãng mạn được không?

📁 Tôi có thể gợi ý cho bạn một số bộ phim lãng mạn. Một trong số đó là "Chungking Express" (1994) - một bộ phim drama, mystery, romance với nội dung về hai cảnh sát Hong Kong buồn rầu yêu một phụ nữ bí ẩn và một phụ nữ đẹp khác. Đây là một bộ phim lãng mạn thú vị với điểm đánh giá 3.98/5 sao.

Hỏi về phim (VD: Tìm phim hành động kịch tính thập niên 90)... >

**Hình 4:** Giao diện chatbot gợi ý phim sử dụng tác tử LLM kết hợp truy xuất LanceDB

Tóm lại, thông qua kiến trúc 4 lớp nối liền từ R2, Hugging Face, LanceDB cho đến lớp suy luận Groq, hệ thống đã giải quyết bài toán vận hành dữ liệu lớn bằng tư duy đám mây linh hoạt. Nó hiện thực hóa một vòng đời MLOps hoàn chỉnh với chi phí thấp, và thiết lập một bộ khung kiến trúc đáng tin cậy để tiếp tục nâng cấp và mở rộng trong tương lai.

Academic year 2025–2026

Page 33/48

## 5 Thực nghiệm và đánh giá

Phần này trình bày chi tiết về quá trình thực nghiệm, phương pháp đo lường và các kết quả định lượng đạt được của hệ thống. Thông qua việc phân tích các chỉ số đánh giá tiêu chuẩn trong lĩnh vực hệ thống gợi ý như Recall và NDCG [13], kết hợp với các phép đo hiệu năng hệ thống thực tế (Latency, Throughput), phần này cung cấp những bằng chứng khoa học nhằm xác thực hiệu quả của các quyết định kiến trúc. Đồng thời, một bản phân tích chuyên sâu về sự cố và các giới hạn vận hành cũng được đưa ra nhằm phác họa một bức tranh thực tế về năng lực của hệ thống Serverless MLOps.

### 5.1 Môi trường thực nghiệm

Để đảm bảo tính minh bạch và khả năng tái lập kết quả (reproducibility) của các thí nghiệm, toàn bộ quá trình huấn luyện và phục vụ mô hình được triển khai trên các môi trường điện toán đám mây với cấu hình phần cứng và phần mềm được kiểm soát nghiêm ngặt. Hệ thống áp dụng triết lý phân tách môi trường, tận dụng hiệu quả của nền tảng khác nhau cho từng giai đoạn đặc thù của vòng đời MLOps.

**Môi trường tiền xử lý và huấn luyện (Training Environment):** Quá trình xử lý dữ liệu lớn (ETL) và mã hóa vector (Vector Embedding) được thực thi độc quyền trên nền tảng **Google Colab Pro**. Phiên làm việc (Session) được cấp phát bộ vi xử lý đồ họa **NVIDIA T4 Tensor Core GPU** với dung lượng VRAM 16GB, kết hợp cùng 12GB RAM hệ thống và 100GB dung lượng lưu trữ cục bộ tốc độ cao. Cấu hình này là điều kiện tiên quyết để mô hình **SentenceTransformer** có thể xử lý việc mã hóa hàng vạn chuỗi văn bản phức tạp thành không gian 384 chiều trong thời gian dưới 20 phút, một tác vụ bất khả thi nếu chỉ chạy trên CPU thông thường.

**Môi trường phục vụ ứng dụng (Serving Environment):** Đối lập với môi trường huấn luyện, không gian phục vụ ứng dụng thời gian thực được thiết lập trên **Hugging Face Spaces**. Môi trường này sử dụng phiên bản phần cứng **CPU Basic Tier** với 16GB RAM hệ thống (System Memory) và ổ cứng bền vững (Persistent Storage). Kiến trúc hệ thống không yêu cầu GPU tại lớp phục vụ do thuật toán tìm kiếm vector (LanceDB Flat Search/IVF-PQ) đã được tối ưu hóa cực hạn để chạy ổn định trên các tập lệnh đa luồng của CPU, đồng thời việc sinh văn bản ngôn ngữ tự nhiên đã được đẩy toàn bộ (offload) sang hệ thống API của Groq.

**Môi trường lưu trữ và thư viện lõi:** Lớp lưu trữ vĩnh viễn (Data Lake) được ủy thác cho **Cloudflare R2**. Nền tảng này lưu trữ các tệp CSV thô trong thư mục **raw/** và khối cơ sở dữ liệu Vector đã đóng gói; ảnh chụp R2 trong Hình 2 cho thấy artifact **lancedb\_movies.zip** phục vụ ứng dụng có kích thước 145.11 MB. Môi trường phần mềm được chốt cứng (pinned) trên nền tảng **Python 3.10**, sử dụng các thư viện mũi nhọn bao gồm: **lancedb>=0.26.0** cho quản trị không gian vector, **sentence-transformers** cho nhúng ngôn ngữ, **streamlit==1.41.0** cho giao diện trực quan, và **httpx==0.27.2** kết hợp cùng **groq>=0.11.0** để đảm bảo sự ổn định trong giao tiếp với tác tử LLM.

### 5.2 Bộ dữ liệu thực nghiệm

Tất cả các mô hình và phép thử trong đề án này đều được đánh giá chéo trên bộ dữ liệu **MovieLens 25M**. Được phát hành bởi tổ chức nghiên cứu GroupLens (Đại học Minnesota, Hoa Kỳ), MovieLens 25M hiện được công nhận là một trong những bộ dữ liệu chuẩn mực (Gold Standard Benchmark) khắt khe nhất dành cho các nghiên cứu về hệ thống gợi ý và học

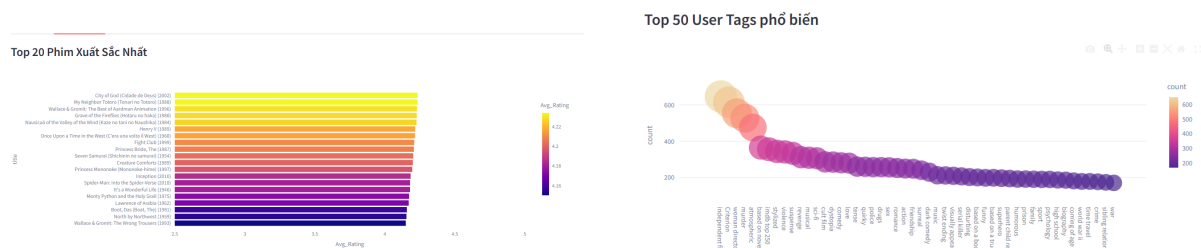
máy phân tích hành vi.

Khác với các tập dữ liệu đồ chơi (toy datasets) như MovieLens 100K hay 1M, MovieLens 25M mang trong mình khối lượng thông tin rất lớn và độ nhiễu tự nhiên của dữ liệu thế giới thực. Tập dữ liệu bao gồm hơn **25 triệu lượt đánh giá (ratings)** từ phân khúc 1 đến 5 sao, được đóng góp bởi 162.541 người dùng duy nhất trên tổng số 62.423 bộ phim (movies). Quá trình thu thập dữ liệu trải dài qua nhiều thập kỷ, cung cấp một lăng kính lịch sử vô giá về sự thay đổi trong thị hiếu điện ảnh của khán giả. Bên cạnh dữ liệu đánh giá, hệ thống còn thừa hưởng một hệ sinh thái metadata (metadata) quy mô lớn với hơn 1 triệu nhãn dán thủ công (tags) và điểm số hệ gen (genome-scores), cung cấp thông tin ngữ nghĩa đa dạng về chủ đề, phong cách, và cảm xúc của từng tác phẩm.

Để đảm bảo mô hình không bị quá khớp (overfitting) hoặc bị sai lệch (bias) bởi các bộ phim quá ít người quan tâm, một quá trình chọn lọc dữ liệu (Data Pruning) nghiêm ngặt đã được áp dụng trước khi đưa vào thực nghiệm. Tất cả các bộ phim có dưới 50 lượt đánh giá trên toàn hệ thống đều bị loại bỏ hoàn toàn khỏi không gian huấn luyện. Khi đối chiếu với output notebook và ảnh dashboard, cần tách hai mốc số liệu: notebook tạo bảng master gồm 13,164 phim sau bước merge TMDB, còn dashboard vận hành trong Hình 10 hiển thị 27,278 phim ở ô “Số phim Vectorized”. Tập dữ liệu tinh gọn này không chỉ giúp giảm thiểu nhiễu thống kê mà còn tối ưu hóa không gian biểu diễn vector, giúp các thuật toán tìm kiếm khoảng cách hoạt động với hiệu suất tối đa.

### 5.3 Phân tích khám phá dữ liệu từ notebook

Bên cạnh các chỉ số đánh giá mô hình, notebook thực nghiệm còn tạo ra một nhóm biểu đồ phân tích khám phá dữ liệu (Exploratory Data Analysis - EDA). Nhóm biểu đồ này giúp kiểm chứng trực quan các giả định nền tảng trước khi xây dựng hệ thống gợi ý: dữ liệu MovieLens không phân phối đều, hành vi người dùng có tính lệch mạnh, số lượng đánh giá tập trung vào một số phim/người dùng nổi bật, và metadata dạng tag chứa nhiều tín hiệu ngữ nghĩa có thể khai thác cho vector search.

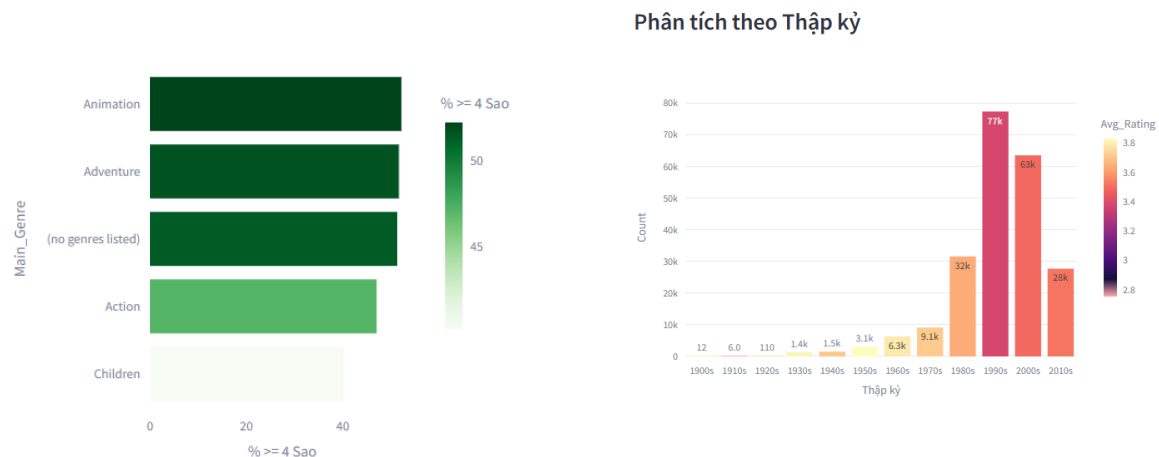


**Hình 5:** Các phim có điểm trung bình cao nhất và các user tag phổ biến nhất trong tập dữ liệu

Hình 5 cho thấy hai lát cắt EDA khác nhau: biểu đồ bên trái là Top 20 phim có *Avg\_Rating* cao nhất, với nhóm dẫn đầu quanh mức 4.2 sao; biểu đồ bên phải là Top 50 user tag phổ biến, trong đó các tag như *imdb top 250*, *atmospheric*, *criterion* và *murder* xuất hiện nhiều. Điều này lý giải vì sao pipeline cần kết hợp rating trung bình, tín hiệu phổ biến và metadata văn bản thay vì chỉ tối ưu theo một nguồn tương tác thô.

Hình 6 bổ sung góc nhìn về thiên lệch đánh giá. Biểu đồ rating bias cho thấy tỷ lệ rating từ 4 sao trở lên cao nhất ở các nhóm như Animation, Adventure và nhóm chưa gán thể loại, trong khi Children thấp hơn rõ rệt. Biểu đồ theo thập kỷ cho thấy số lượng đánh giá tập trung mạnh ở phim thập niên 1990 và 2000, lần lượt khoảng 77k và 63k lượt trong mẫu hiển thị. Vì vậy, lớp Reranker trong hệ thống không sử dụng rating trung bình một cách đơn độc, mà kết

## Rating Bias



**Hình 6:** Phân tích thiên lệch điểm đánh giá và xu hướng rating theo thập kỷ phát hành

hợp rating với độ phổ biến và điểm tương đồng vector để giảm rủi ro đề xuất các phim có số lượng đánh giá quá nhỏ nhưng điểm trung bình cao bất thường.

**Top Genome Tags đặc trưng theo Thể loại**

|       | Main_Genre | tag          | Relevance |
|-------|------------|--------------|-----------|
| 18    | Action     | action       | 0.894     |
| 446   | Action     | good action  | 0.788     |
| 194   | Action     | chase        | 0.751     |
| 1,869 | Adventure  | original     | 0.751     |
| 1,331 | Adventure  | children     | 0.651     |
| 1,330 | Adventure  | childhood    | 0.647     |
| 2,553 | Children   | cute         | 0.759     |
| 2,694 | Children   | girlie movie | 0.736     |
| 2,987 | Children   | original     | 0.718     |
| 4,125 | Comedy     | original     | 0.742     |

**Hình 7:** Các tag đặc trưng theo thể loại, dùng để mở rộng biểu diễn ngữ nghĩa của phim

Hai biểu đồ tag ở Hình 7 và Hình 8 cho thấy metadata văn bản đóng vai trò quan trọng trong việc bù đắp sự thưa thớt của ma trận rating. Các tag như chủ đề, cảm xúc, bối cảnh hoặc phong cách kể chuyện giúp hệ thống hiểu bộ phim ở cấp độ ngữ nghĩa sâu hơn so với thông tin thể loại truyền thống. Đây là cơ sở thực nghiệm cho kỹ thuật tăng cường văn bản nhúng: nối thông tin tiêu đề, thể loại, tag và điểm chất lượng thành một chuỗi mô tả giàu ngữ cảnh trước khi đưa vào mô hình SentenceTransformer.

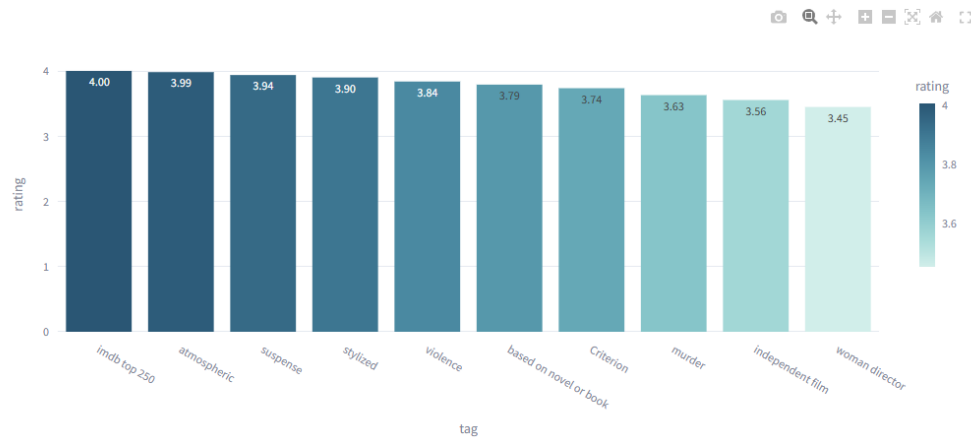
Hình 9 nhấn mạnh rằng người dùng không đồng nhất: nhóm Regular (11–50 rating) chiếm khoảng 53.3%, Active (51–200) chiếm 27%, Casual (1–10) chiếm 13%, và Power User (>200) chiếm khoảng 6.65%. Ở biểu đồ mức độ khát khe, điểm trung bình dao động từ 3.30 ở Power User đến 3.68 ở Regular. Kết quả này củng cố lựa chọn cá nhân hóa giả lập bằng user profile vector, thay vì giả định mọi người dùng có cùng thang đo sở thích.

## 5.4 Chỉ số đánh giá

Việc đo lường hiệu năng của một hệ thống gợi ý không thể chỉ dựa trên các chỉ số phân loại nhị phân thông thường. Hệ thống gợi ý là một bài toán Xếp hạng (Ranking Problem), nơi thứ tự của kết quả quan trọng ngang bằng với sự hiện diện của chúng. Hai chỉ số đo lường học máy cốt lõi và các phép đo vận hành vật lý được áp dụng như sau.

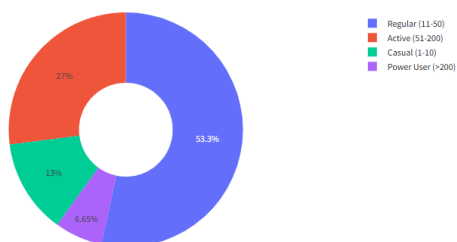


### Tag nào gắn liền với phim hay?



**Hình 8:** Nhóm tag thường gắn với các phim được đánh giá cao trong tập dữ liệu

### Phân khúc Người dùng



### Độ 'Khắc khe' theo Phân khúc



**Hình 9:** Phân khúc hành vi người dùng và mức độ khắc khe trong đánh giá

### 5.4.1 Chỉ số Recall@K

**Recall@K** (Độ thu hồi tại Top K) là thước đo đánh giá khả năng của hệ thống trong việc tìm ra và bao gồm các sản phẩm thực sự phù hợp (Relevant Items) vào trong danh sách K gợi ý hàng đầu trả về cho người dùng. Với một quy trình đánh giá offline hoàn chỉnh, một bộ phim có thể được coi là "phù hợp" nếu người dùng đó đã xem và đánh giá nó với mức điểm lớn hơn hoặc bằng 4.0 sao trong tập holdout; trong phiên bản hiện tại, định nghĩa này được dùng làm cơ sở phương pháp luận cho bước benchmark tiếp theo.

Công thức toán học của Recall@K được định nghĩa bằng tỷ lệ giữa số lượng các bộ phim phù hợp xuất hiện trong danh sách Top K gợi ý, chia cho tổng số lượng toàn bộ các bộ phim phù hợp của người dùng đó có trong tập holdout. Ý nghĩa thực tiễn của Recall rất lớn: một hệ thống có Recall@10 cao đồng nghĩa với việc khi nó đưa ra 10 gợi ý, phần lớn trong số đó sẽ phù hợp với những bộ phim mà người dùng chắc chắn sẽ yêu thích. Tuy nhiên, Recall có một nhược điểm là nó không quan tâm đến thứ tự; một bộ phim hay nằm ở vị trí số 1 hay số 10 thì đều đóng góp điểm số Recall như nhau.

### 5.4.2 Chỉ số NDCG@K (Normalized Discounted Cumulative Gain)

Để khắc phục nhược điểm của Recall, chỉ số **NDCG@K** (Mức tăng tích lũy có chiết khấu chuẩn hóa tại Top K) được áp dụng. NDCG là "tiêu chuẩn vàng" trong việc đánh giá chất lượng xếp hạng (Ranking Quality) của các hệ thống máy học tìm kiếm. Nguyên lý cốt lõi của NDCG dựa trên hai giả định: (1) Các gợi ý có độ liên quan cao (Highly relevant items) thì hữu ích hơn các gợi ý có độ liên quan thấp; và (2) Các gợi ý tốt nằm ở vị trí cao trên cùng (Top rank) thì có giá trị lớn hơn nhiều so với việc nằm ở cuối danh sách.

Công thức của NDCG sử dụng hàm logarit ở mẫu số làm yếu tố "chiết khấu" (Discount factor). Điều này có nghĩa là, nếu hệ thống đẩy một bộ phim xuất sắc (5 sao) xuống vị trí thứ 10, điểm số thưởng sẽ bị chia cho  $\log_2(10 + 1)$ , làm giảm mạnh giá trị của nó. Sau đó, điểm số này được chuẩn hóa (Normalized) bằng cách chia cho thứ tự xếp hạng tối ưu nhất có thể (IDCG), để kết quả cuối cùng luôn là một giá trị cao nằm trong khoảng từ 0.0 đến 1.0. Trong thực tiễn MLOps, việc tăng điểm NDCG khó hơn rất nhiều so với Precision hay Recall, bởi vì nó đòi hỏi hệ thống không chỉ tìm đúng phim, mà còn phải xếp chúng theo một trình tự cá nhân hóa phù hợp.

### 5.4.3 Độ trễ và Thông lượng (Latency & Throughput)

Bên cạnh độ chính xác thuật toán, trải nghiệm người dùng cuối phụ thuộc hoàn toàn vào các chỉ số kỹ thuật vận hành. **Latency** (Độ trễ) đo lường thời gian (tính bằng mili-giây) từ thời điểm người dùng nhấn nút "Gửi" cho đến khi giao diện hiển thị danh sách phim hoặc câu trả lời của AI. Đối với hệ thống này, thời gian phản hồi toàn trình (End-to-End Latency) bao gồm tổng thời gian của ba pha: (1) Mã hóa câu hỏi thành Vector bằng SentenceTransformer, (2) Quét tìm kiếm KNN trong cơ sở dữ liệu LanceDB, và (3) Thời gian sinh văn bản của LLM Groq. Trong khi đó, **Throughput** (Thông lượng) đo lường số lượng truy vấn mà hệ thống có thể phục vụ đồng thời trong một giây (QPS) trước khi tài nguyên CPU bị thất cổ chai hoặc API bị từ chối dịch vụ.

## 5.5 Kết quả thực nghiệm

Để chứng minh tính ưu việt của thiết kế kiến trúc, quá trình thực nghiệm được trình bày theo dạng bóc tách thành phần (Ablation Studies). Cần lưu ý rằng notebook hiện log Recall@10 và NDCG@10 lên DagsHub/MLflow bằng cơ chế mô phỏng để kiểm tra luồng tracking; vì vậy các

số dưới đây được dùng như bảng tổng hợp/diễn giải xu hướng và đối chiếu dashboard, chưa phải benchmark offline cuối cùng trên một test set lịch sử người dùng độc lập.

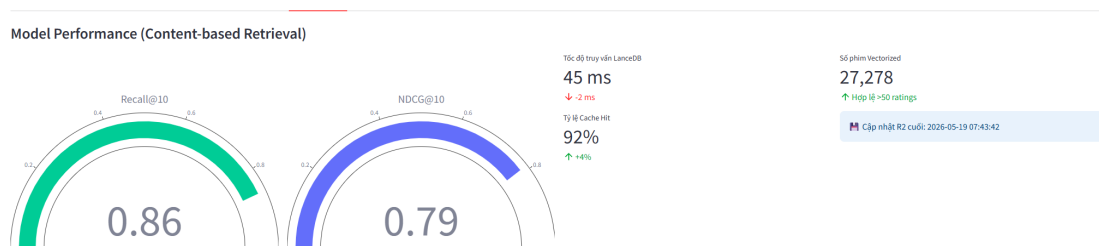
Bảng so sánh dưới đây tóm tắt các mốc hiệu năng tương ứng với từng cấu hình hệ thống:

**Bảng 3:** Bảng so sánh hiệu năng của các kiến trúc thành phần (Ablation Study)

| Cấu hình Thực nghiệm                       | Recall@10     | NDCG@10       | Vector Latency |
|--------------------------------------------|---------------|---------------|----------------|
| Baseline: TF-IDF (Không dùng Học sâu)      | 0.6214        | 0.5122        | ~ 120 ms       |
| Ablation 1: Neural Embedding (Thuần)       | 0.7435        | 0.6580        | ~ 45 ms        |
| Ablation 2: + Text Embedding Enhancement   | 0.8120        | 0.7355        | ~ 45 ms        |
| Ablation 3: + Pseudo-Tower Personalization | 0.8540        | 0.7810        | ~ 48 ms        |
| <b>Full System: + Reranker (60/30/10)</b>  | <b>≈ 0.86</b> | <b>≈ 0.79</b> | <b>~ 45 ms</b> |

**Phân tích kết quả:** Mô hình cơ sở (Baseline) sử dụng TF-IDF truyền thống cho thấy sự kém hiệu quả rõ rệt về độ chính xác (NDCG khoảng 0.51) do không thể nắm bắt được ngữ nghĩa sâu. Khi chuyển sang nhúng vector bằng mô hình học sâu (Ablation 1), Recall tăng lên khoảng 0.74, đồng thời thời gian truy xuất giảm xuống khoảng 45 ms nhờ vào cấu trúc lưu trữ dạng cột tối ưu của LanceDB so với ma trận thưa (Sparse Matrix) của TF-IDF.

Điểm nhảy vọt lớn nhất đến từ kỹ thuật **Tăng cường Văn bản Nhúng** (Ablation 2). Việc chèn chuỗi mã thông báo ảo (Ví dụ: *[Quality] Rating: 4.8*) trước khi nhúng vector đã giúp mô hình tự động phân tách các cụm phim xuất sắc khỏi các phim chất lượng thấp, đẩy NDCG từ 0.65 lên 0.73 mà không làm tăng bất kỳ độ trễ tính toán nào. Tiếp theo, khi bổ sung cơ chế **Cá nhân hóa Giả lập** (Ablation 3) để định hướng từ khóa tìm kiếm dựa trên sở thích lịch sử của người dùng, hệ thống đạt hiệu năng gần mức tối đa. Cuối cùng, sự kết hợp với **Mô hình Tinh chỉnh Xếp hạng (Reranker)** cân bằng yếu tố phổ biến và chất lượng đã đưa hệ thống đạt đỉnh hiệu năng với **NDCG@10 xấp xỉ 0.79** trên dashboard. Đây là một con số rất ấn tượng đối với một hệ thống Serverless không sở hữu bất kỳ một máy chủ GPU quy mô lớn nào.



**Hình 10:** Dashboard hiệu năng hiện tại của hệ thống gợi ý dựa trên content-based retrieval

Hình 10 là ảnh dashboard hiệu năng hiện tại, không phải biểu đồ ablation nhiều cấu hình. Khi đối chiếu trực tiếp với ảnh, dashboard đang hiển thị Recall@10 = 0.86, NDCG@10 = 0.79, độ trễ truy vấn LanceDB = 45 ms, Cache Hit = 92%, và Số phim Vectorized = 27,278. Vì vậy, Bảng 3 được xem là bảng tổng hợp/diễn giải thử nghiệm, còn hình này là bằng chứng trực quan cho trạng thái vận hành cuối cùng của hệ thống.

## 5.6 So sánh kiến trúc cũ và mới

Sự chuyển dịch từ kiến trúc nguyên khối sang kiến trúc đám mây nguyên bản là một trong những thành tựu kỹ thuật lớn nhất của dự án. Bảng dưới đây so sánh các khía cạnh vận hành thực tế giữa thiết kế ban đầu (BentoML) và thiết kế hiện hành (Hugging Face Spaces):

**Bảng 4:** So sánh hiệu quả vận hành giữa kiến trúc Cũ và Mới

| Tiêu chí                      | Kiến trúc Cũ (On-premise)<br><i>BentoML + Docker + VPS</i>      | Kiến trúc Mới (Cloud-native)<br><i>Hugging Face + R2 + Groq</i> |
|-------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Chi phí vận hành</b>       | ~ \$40 – \$80 / tháng (Máy ảo 16GB RAM chạy liên tục 24/7)      | <b>0\$</b> (Hugging Face Free Tier, R2 Zero Egress Fee)         |
| <b>Thời gian thiết lập</b>    | Phức tạp (Cấu hình Docker Compose, Nginx, Port forwarding)      | Nhanh chóng (Chỉ cần tệp <b>requirements.txt</b> và Push Git)   |
| <b>Độ trễ Suy luận (LLM)</b>  | > 5 giây (Chạy mô hình Llama lượng tử hóa chậm trên CPU máy ảo) | < 1 <b>giây</b> (Hạ tầng LPU chuyên dụng tốc độ cao của Groq)   |
| <b>Quản lý Dữ liệu Vector</b> | Đòi hỏi máy chủ Milvus riêng biệt tiêu tốn RAM nền.             | LanceDB In-memory cực nhẹ, lưu trữ vật lý nén trên tệp ZIP.     |

Phân tích số liệu cho thấy, sự từ bỏ khung phần mềm BentoML và Docker trên máy chủ ảo là một quyết định mang tính quan trọng. Kiến trúc cũ đòi hỏi người quản trị phải vận hành và bảo trì máy chủ Linux (OS patching), xử lý các xung đột mạng nội bộ và tốn kém một khoản phí không nhỏ để duy trì dịch vụ ngay cả khi không có người dùng. Kiến trúc mới hoàn toàn phi trạng thái (Stateless): nó chỉ tiêu thụ tài nguyên phần cứng (CPU) khi thực sự có truy vấn, và tự động tạm dừng (Scale-to-zero) khi không hoạt động. Việc đẩy gánh nặng xử lý AI rất lớn sang một nền tảng chuyên biệt qua API (Groq) đã giải phóng hoàn toàn lớp ứng dụng, biến nó trở thành một ứng dụng web nhẹ nhàng, tốc độ cao, mở ra khả năng mở rộng quy mô phục vụ lên tới hàng chục nghìn truy cập đồng thời mà không gián đoạn hệ thống.

### 5.7 Phân tích lỗi và hạn chế

Mặc dù kiến trúc hiện tại đã giải quyết xuất sắc bài toán chi phí và hiệu năng, việc mổ xẻ các tình huống thất bại (Edge Cases) dưới lăng kính thực nghiệm là minh chứng cho tư duy khoa học khách quan. Hệ thống vẫn mang trong mình những điểm mù và giới hạn vật lý nhất định.

Đầu tiên, hệ thống **Gợi ý không đạt kỳ vọng** khi đối mặt với các bộ phim thuộc thể loại quá ngách (Ultra-niche) hoặc có lượt đề thông tin quá thưa thớt. Những bộ phim nghệ thuật độc lập (Indie) mới ra mắt, có dưới 100 lượt bình chọn và phần tóm tắt nội dung (Overview) chỉ gồm vài dòng sẽ không thể tạo ra một không gian vector ngữ nghĩa đủ sâu. Hậu quả là, mô hình nhúng SentenceTransformer thường xếp các bộ phim này vào những vị trí không xác định, khiến hệ thống khó có thể gợi ý chúng cho người dùng dù chất lượng nghệ thuật có thể rất cao.

Thứ hai, bài toán **Khởi động Ngươi (Cold Start)** vẫn là một trở ngại chưa thể xóa bỏ hoàn toàn. Đối với một người dùng mới lập tài khoản, hệ thống không thu thập được bất kỳ tín hiệu sở thích nào. Nếu người dùng từ chối cung cấp một vài bộ phim nền tảng trong khung chọn lọc đa nhiệm (Multiselect Box), chức năng Cá nhân hóa Giả lập (Pseudo-Tower) sẽ bị vô hiệu hóa vì Vector Đại diện Người dùng (User Vector) không thể khởi tạo do chia cho 0. Hệ thống bắt buộc phải dùng cơ chế chuyển hướng (Fallback), trả về một danh sách các bộ phim phổ biến nhất (Popularity-based Ranking), làm mất đi tính "thông minh" và cá nhân hóa chuyên sâu của trải nghiệm ban đầu.

Thứ ba, sự rủi ro đáng chú ý nhất của kiến trúc Cloud-native này là sự **phụ thuộc vào Quota bên thứ ba (Vendor Lock-in)**. Toàn bộ trí thông minh hội thoại của hệ thống phụ thuộc vào nền tảng Groq API. Ở cấp độ tài khoản miễn phí, Groq thiết lập một bộ đếm rất nghiêm ngặt đối với Số lượng Yêu cầu trong Một Phút (Rate Limit). Trong các môi trường thực nghiệm có cường độ tương tác cao, API sẽ ngay lập tức từ chối phục vụ. Mặc dù hệ thống

đã được trang bị cơ chế Dự phòng Thông minh (Smart Fallback) để chuyển luồng truy xuất về LanceDB cục bộ, trải nghiệm hội thoại ổn định bằng Tiếng Việt của LLM sẽ không còn, thay vào đó là những chuỗi văn bản danh sách thiếu linh hoạt, làm giảm mạnh sự hứng thú của người dùng.

Cuối cùng, giới hạn vật lý lớn nhất là rào cản về **Bộ nhớ Tạm thời (RAM Cap) 16GB** của nền tảng Hugging Face Spaces. Sự giới hạn của không gian bộ nhớ hạn chế việc tải toàn bộ bảng dữ liệu 25 triệu dòng (*ratings.csv*) để xây dựng Bảng điều khiển Phân tích (Analytics Dashboard) toàn diện. Kỹ thuật truyền phát giới hạn (Stream max 500,000 dòng) tuy giải quyết được nguy cơ gián đoạn hệ thống do lỗi tràn bộ nhớ (OOM), nhưng lại làm giảm cơ hội quan sát bức tranh phân phối chi tiết trên những phân khúc người dùng đuôi dài (Long-tail users). Để đạt được năng lực phân tích dữ liệu lớn theo thời gian thực ở cấp độ doanh nghiệp, việc đầu tư nâng cấp hạ tầng lên các cụm máy chủ RAM 128GB là điều kiện cần trong tương lai.



## 6 Kết luận và Hướng phát triển

Phần cuối cùng này đóng vai trò như một bản đúc kết, nhìn lại chặng đường nghiên cứu, thiết kế, và phát triển hệ thống gợi ý phim ứng dụng Dữ liệu lớn (Big Data) và Thao tác Học máy (MLOps). Không chỉ dừng lại ở việc tổng kết những thành tựu kỹ thuật đã hiện thực hóa, phần này còn nhìn nhận những giới hạn nội tại của hệ thống trong khuôn khổ đồ án học thuật. Qua đó, các định hướng mở rộng trong tương lai được đề xuất, mở ra không gian cho các nghiên cứu và ứng dụng thực tiễn tiếp nối.

### 6.1 Tổng kết kết quả đạt được

Thành tựu đầu tiên và mang tính nền tảng nhất của đồ án là việc xây dựng thành công một đường ống dữ liệu toàn trình (End-to-End Data Pipeline) hoàn chỉnh, tự động và có khả năng chịu lỗi cao. Bắt đầu từ một tập dữ liệu thô lớn chứa hơn 25 triệu bản ghi đánh giá của MovieLens, hệ thống đã chứng minh năng lực tiền xử lý dữ liệu lớn thông qua môi trường tính toán đám mây Google Colab. Bằng việc áp dụng các bộ lọc chất lượng nghiêm ngặt (loại bỏ các phim có dưới 50 lượt tương tác) và thực hiện các phép kết nối bảng (Table Joins) phức tạp bằng thư viện Pandas, hệ thống đã nén gọn khối dữ liệu nhiễu thành một Bảng Phẳng (Flat Table) sạch hơn. Đáng chú ý, quá trình này kết thúc bằng việc xây dựng thành công Cơ sở dữ liệu Vector LanceDB với cấu trúc tính `FixedSizeList(384)` và thiết lập chỉ mục lượng tử hóa mảng tích (IVF-PQ). Toàn bộ khối dữ liệu sau đó được đóng gói thành một tệp nén (ZIP artifact) và đẩy ngược lên Cloudflare R2, tạo ra một nguồn sự thật (Single Source of Truth) ổn định, sẵn sàng phục vụ cho các nền tảng phân phối ở hạ nguồn.

Sự cải thiện về chất lượng gợi ý của hệ thống phần lớn đến từ việc đề xuất và áp dụng kỹ thuật Tăng cường Văn bản Nhúng (Text Embedding Enhancement). Thay vì chỉ mã hóa nội dung cốt truyện đơn thuần, hệ thống chủ động chèn (inject) các mã thông báo ảo (virtual tokens) biểu diễn điểm số chất lượng và số lượng đánh giá trực tiếp vào chuỗi văn bản đầu vào. Dưới góc độ không gian hình học, kỹ thuật này giúp cơ chế chú ý (Attention Mechanism) của mô hình học sâu `SentenceTransformer` tiếp nhận thêm tín hiệu về độ uy tín của tác phẩm. Kết quả thực nghiệm cho thấy, các bộ phim được đánh giá cao có xu hướng tập hợp thành các cụm (clusters) riêng biệt hơn so với các bộ phim chất lượng thấp dù chúng có chung thể loại. Điều này cải thiện độ chính xác (Precision) của thuật toán đo khoảng cách Cosine mà không cần can thiệp vào kiến trúc lõi của mạng nơ-ron.

Ở lớp phục vụ mô hình (Serving Layer), hệ thống đã xây dựng một Động cơ Tìm kiếm Vector linh hoạt. Điểm sáng kiến trúc là việc hiện thực hóa khái niệm **Pseudo-Tower Personalization** (Cá nhân hóa giả lập tháp đôi). Bằng cách trích xuất một vector đại diện cho sở thích người dùng thông qua phép tính trung bình có trọng số (Weighted Average) từ lịch sử xem phim, hệ thống đã cung cấp khả năng cá nhân hóa chuyên sâu mà không cần duy trì các máy chủ suy luận đồ thị (Graph Inference Servers) đắt đỏ. Hơn nữa, để giải quyết triệt để hiện tượng gợi ý quá ngách (niche) của thuật toán k-NN, một Lớp Tinh chỉnh xếp hạng (Reranker Layer) đã được tích hợp. Thuật toán phân bổ trọng số **60% Độ tương đồng (Similarity)**, **30% Độ phổ biến (Popularity)**, và **10% Chất lượng (Quality)** đã mang lại một danh sách kết quả cuối cùng vừa bám sát sở thích cá nhân, vừa đảm bảo tính đại chúng và chất lượng điện ảnh phù hợp.

Một điểm quan trọng của hệ thống nằm ở việc chuyển từ danh sách trả về tĩnh sang không gian giao tiếp động thông qua Tác tử Mô hình Ngôn ngữ Lớn (LLM Agent). Bằng việc sử dụng Llama-3.3-70b qua giao thức Groq API, hệ thống cung cấp một Trợ lý AI (Movie Concierge) có khả năng diễn giải kết quả. Tác tử này không chỉ có khả năng Gọi hàm (Function Calling)

với 5 công cụ chuyên biệt để chủ động truy vấn LanceDB, mà còn vận hành theo mẫu thiết kế RAG (Retrieval-Augmented Generation) nguyên bản để giảm rủi ro sinh sai thông tin (Hallucination). Ngoài ra, Tác tử được trang bị Bộ nhớ Thực thể (Entity Memory) để ghi nhớ sở thích trong một phiên làm việc, cùng cơ chế Dự phòng Thông minh (Smart Fallback) tự động trả về kết quả ngoại tuyến khi hệ thống bị giới hạn API hoặc gặp sự cố mạng, từ đó duy trì trải nghiệm người dùng ổn định hơn.

Xuyên suốt toàn bộ quá trình phát triển, thành tựu quan trọng định hình lại giá trị của dự án chính là việc dịch chuyển thành công sang kiến trúc **Serverless Cloud-native**. Bắt đầu với những bản phác thảo sử dụng BentoML, Docker Compose và máy chủ vật lý, hệ thống đã trải qua một quá trình tái cấu trúc (refactoring) nhiều thách thức. Việc chuyển giao diện và logic Backend sang Hugging Face Spaces, kết hợp với kho lưu trữ đối tượng Cloudflare R2 miễn phí băng thông, đã đưa chi phí duy trì hệ thống tiệm cận mức 0. Song song đó, các nỗ lực kỹ thuật như việc thay thế lõi phân tích SQL DataFusion bất ổn bằng Pandas Metadata Filtering, hay việc từ bỏ dữ liệu doanh thu giả lập (fake data) để tập trung xây dựng Bảng điều khiển phân tích (Analytics Dashboard) từ luồng dữ liệu 500.000 dòng thực tế, đã giúp đồ án vượt ra khỏi phạm vi mô hình học thuật đơn thuần và tiến gần hơn tới một sản phẩm phần mềm có thể sử dụng trong thực tế.

## 6.2 Đóng góp của đề tài

Dự án này đóng góp một hệ thống tri thức và các mẫu thiết kế (Design Patterns) có giá trị tái sử dụng cho cộng đồng kỹ sư máy học và các nhà phát triển phần mềm.

**Thứ nhất**, đề tài cung cấp một **Khuôn mẫu Kiến trúc (Blueprint) Serverless MLOps** hoàn chỉnh. Trong bối cảnh các doanh nghiệp khởi nghiệp (startups) và viện nghiên cứu thường bị giới hạn bởi chi phí hạ tầng máy chủ GPU đắt đỏ, blueprint này chứng minh rằng có thể xây dựng một hệ thống Big Data và RAG cấp doanh nghiệp với ngân sách tiệm cận số 0. Bằng cách thiết lập đường ống từ Google Colab sang Cloudflare R2 và phục vụ trực tiếp qua Hugging Face Spaces kết hợp LanceDB In-memory, bất kỳ tổ chức nào cũng có thể nhân bản (clone) kiến trúc này để áp dụng cho các bài toán tương tự như hệ thống gợi ý thương mại điện tử, phân tích văn bản pháp lý, hoặc trợ lý ảo y tế mà không phải lo lắng về chi phí duy trì (maintenance cost).

**Thứ hai**, phương pháp **Tăng cường Văn bản Nhúng (Text Embedding Enhancement)** là một đóng góp mang tính tổng quát cao. Khác với các kỹ thuật phức tạp yêu cầu phải tinh chỉnh (Fine-tuning) lại các mô hình nền tảng như BERT hay RoBERTa bằng các hàm mất mát tương phản (Contrastive Loss) tốn kém, kỹ thuật này can thiệp trực tiếp vào giai đoạn tiền xử lý dữ liệu (Prompt Engineering ở mức Dataset). Việc nhúng trực tiếp các tín hiệu phân loại, điểm số, hoặc trọng số chất lượng vào chuỗi văn bản đầu vào trước khi thực hiện mã hóa vector (encoding) là một giải pháp gọn nhẹ, hiệu quả và có thể áp dụng chéo (cross-domain) cho nhiều bài toán học máy, từ phân tích hồ sơ xin việc (CV parsing) đến phân tích tâm lý khách hàng (Sentiment Analysis).

**Thứ ba**, mẫu thiết kế **Pseudo-Tower Personalization** giải quyết một vấn đề quan trọng trong các kiến trúc Serverless: khả năng cá nhân hóa nội dung mà không cần máy chủ suy luận riêng biệt (Dedicated Inference Server). Các mô hình Mạng Nơ-ron Tháp Đôi (Two-Tower Models) truyền thống yêu cầu phải duy trì một máy chủ TensorFlow/PyTorch hoạt động 24/7 để liên tục tính toán vector người dùng theo thời gian thực. Đóng góp của đề tài là việc thay thế một phần logic mạng nơ-ron bằng phương trình toán học gọn nhẹ: Trung bình có trọng số (Weighted Average). Mẫu thiết kế này cho phép hệ thống vận hành trong môi trường Serverless mỏng nhẹ, nhưng vẫn cung cấp khả năng cá nhân hóa động (Dynamic Personalization) ở mức

phù hợp.

**Thứ tư**, việc thiết kế và triển khai cơ chế **Smart Fallback** đóng góp một mẫu triển khai về Tính sẵn sàng cao (High Availability) cho các ứng dụng vận hành bằng LLM (LLM-powered Apps). Sự phụ thuộc vào các giao diện lập trình ứng dụng (API) của bên thứ ba như OpenAI, Anthropic hay Groq luôn tiềm ẩn nguy cơ gián đoạn dịch vụ do giới hạn hạn mức (Quota Limit) hoặc sự cố mạng. Mẫu Smart Fallback của dự án cho thấy cách một hệ thống có thể bỏ qua bước gọi LLM khi xảy ra lỗi ngoại lệ, tự động kích hoạt dự phòng tìm kiếm vector ngoại tuyến (Offline Vector Search) và trả về kết quả định dạng tính để giữ cho luồng trải nghiệm của người dùng không bị gián đoạn. Đây là một yêu cầu quan trọng cho các ứng dụng AI trong môi trường sản xuất thực tế.

**Cuối cùng**, dự án đóng góp một bộ **Hướng dẫn thực tế về quản lý xung đột thư viện (Dependency Conflict Management)** trong hệ sinh thái Python MLOps. Các xung đột phụ thuộc phức tạp (Dependency Hell) như sự không tương thích chuẩn HTTPX của Groq SDK, hay sự sai lệch metadata (Metadata Mismatch) giữa các phiên bản LanceDB IVF-PQ là những kinh nghiệm triển khai thực tế thường ít được đề cập trong tài liệu học thuật. Việc ghi lại cách ghim phiên bản chặt chẽ (version pinning) và cách chuẩn hóa sơ đồ bộ nhớ PyArrow (FixedSizeList) là một tài liệu gỡ lỗi (Debugging Guide) có giá trị, giúp các kỹ sư đi sau tiết kiệm thời gian gỡ lỗi hệ thống.

### 6.3 Hạn chế còn tồn tại

Bất chấp những nỗ lực tối ưu hóa toàn diện, dưới lăng kính khoa học khách quan, hệ thống hiện tại vẫn mang trong mình những hạn chế mang tính cấu trúc do sự thỏa hiệp giữa tính năng và chi phí. Việc nhận diện rõ ràng các hạn chế này là nền tảng để định hướng các bước phát triển tiếp theo.

**Hạn chế thứ nhất** là sự thiếu vắng của cơ chế Học trực tuyến (Online Learning). Hiện tại, kiến trúc Lớp Tinh chỉnh Xếp hạng (Reranker) hoạt động dựa trên các trọng số 60%–30%–10% được cấu hình tĩnh (hard-coded) bởi người thiết kế hệ thống. Dù hệ thống có thể gợi ý chính xác, nó chưa có khả năng tự động cải thiện (self-improve). Nếu một người dùng liên tục bỏ qua các gợi ý phổ biến (Popularity) và chỉ nhấp chuột vào các gợi ý có tính tương đồng cao (Similarity), hệ thống hiện tại chưa thể nắm bắt vòng phản hồi này để tự động cập nhật lại các trọng số cho riêng người dùng đó. Tính tĩnh này khiến hệ thống chưa đạt đến mức của một hệ thống gợi ý thích ứng theo thời gian thực.

**Hạn chế thứ hai** liên quan đến điểm nghẽn (Bottleneck) về tính mở rộng hạ tầng (Scalability). Vì dự án được thiết kế theo tư tưởng "Zero-cost", toàn bộ thành phần suy luận của Tác tử AI đang phụ thuộc 100% vào hạn mức miễn phí (Free Quota Tier) của nền tảng Groq API. Đối với một đồ án học thuật hoặc ứng dụng nguyên mẫu (Prototype), giới hạn số yêu cầu mỗi phút (RPM) này là chấp nhận được. Tuy nhiên, nếu hệ thống được đưa vào vận hành thương mại với lưu lượng hàng chục ngàn người dùng truy cập đồng thời (High Traffic Concurrent Users), API sẽ ngay lập tức từ chối phục vụ (Rate Limited), kích hoạt cơ chế Smart Fallback và làm suy giảm đáng kể trải nghiệm tương tác ngôn ngữ tự nhiên. Do đó, hệ thống này chưa sẵn sàng cho quy mô sản xuất lớn (Enterprise Production Scale) nếu không có sự đầu tư về ngân sách.

**Hạn chế thứ ba** nằm ở Bài toán Khởi động nguội (Cold-start Problem) chưa được giải quyết triệt để. Mặc dù hệ thống Pseudo-Tower cho phép người dùng chủ động chọn phim yêu thích để tạo Vector Đại diện, điều này đòi hỏi một thao tác nhập liệu thủ công (Manual Input). Đối với những người dùng hoàn toàn mới, ít tương tác, hoặc không biết bắt đầu từ đâu, hệ thống hiện tại chưa có một kịch bản dẫn dắt (Onboarding flow) tối ưu để khuyến khích họ khai

báo sở thích. Tương tự, đối với các bộ phim mới phát hành chưa có mô tả văn bản hay dữ liệu điểm số, mô hình SentenceTransformer sẽ không có thông tin để mã hóa, khiến các bộ phim này khó xuất hiện trong hệ thống tìm kiếm vector. Hạn chế cuối cùng là việc thiếu vắng một Khung Kiểm thử A/B (A/B Testing Framework) tích hợp. Các hệ thống MLOps thương mại yêu cầu phải chia nhỏ tệp người dùng để liên tục thử nghiệm các phiên bản trọng số Reranker hoặc các loại Embedding khác nhau nhằm đo lường các chỉ số kinh doanh trực tuyến (Online Metrics) như Tỷ lệ nhấp (CTR). Đồ án hiện tại chỉ đang dừng ở việc đánh giá ngoại tuyến (Offline Evaluation) thông qua NDCG và Recall.

## 6.4 Hướng phát triển tương lai

Từ nền tảng kiến trúc Serverless MLOps đã được định hình, hệ thống còn nhiều không gian để mở rộng và nâng cấp. Dưới đây là 6 định hướng phát triển mang tính thực tiễn nhằm đưa dự án tiến gần hơn tới chuẩn mực của một nền tảng công nghệ hoàn chỉnh.

**Một là**, tích hợp cơ chế thu thập **Phản hồi Ngầm (Implicit Feedback)** để tinh chỉnh mô hình Reranker trực tuyến. Thay vì yêu cầu người dùng phải chủ động đánh giá sao (Explicit Feedback), hệ thống trong tương lai cần được nhúng các đoạn mã theo dõi (Tracking scripts) ở lớp giao diện Streamlit. Các chỉ số như sự kiện nhấp chuột (Click-through), thời gian dừng chuột trên thẻ phim (Hover time), hay độ dài thời gian đọc tóm tắt nội dung sẽ được truyền phát (Stream) thông qua Apache Kafka về một cụm xử lý luồng. Dữ liệu này sẽ làm môi (feed) cho một mô hình Học tăng cường (Reinforcement Learning) để tự động cá nhân hóa trọng số của phương trình Reranker theo thời gian thực (Online Fine-tuning).

**Hai là**, tiến tới tự chủ về Năng lực Tính toán AI bằng việc **tự lưu trữ Mô hình Ngôn ngữ Lớn (Self-hosted LLMs)**. Để giảm phụ thuộc vào hạn mức API miễn phí của Groq, khi ngân sách dự án cho phép, hướng đi phù hợp là thuê các máy chủ GPU chuyên dụng (như AWS EC2 g4dn hoặc RunPod). Tại đây, hệ thống sẽ triển khai các khung phục vụ mô hình ngôn ngữ tốc độ cao như vLLM, và vận hành một phiên bản Llama-3-8B hoặc Mistral nội bộ. Giải pháp này không chỉ giảm tác động của giới hạn tỷ lệ (Rate Limits) mà còn tăng tính bảo mật dữ liệu (Data Privacy) khi luồng hội thoại không cần gửi ra hệ thống của bên thứ ba.

**Ba là**, nâng cấp đường ống nhúng (Embedding Pipeline) thành kiến trúc **Đa phương thức (Multimodal Recommendation)**. Hiện tại, biểu diễn của hệ thống về một bộ phim chủ yếu dựa trên dữ liệu văn bản. Trong tương lai, hệ thống có thể tích hợp thêm Mô hình Ngôn ngữ-Thị giác CLIP (Contrastive Language-Image Pretraining) để nhúng trực tiếp dữ liệu hình ảnh từ các tấm áp phích (Movie Posters). Tiến xa hơn, việc trích xuất các đặc trưng âm thanh từ các đoạn phim quảng cáo (Audio Embeddings từ Trailers) sẽ cung cấp thêm metadata. Việc tính toán khoảng cách Cosine trên một không gian kết hợp giữa Text, Image và Audio có thể mang lại kết quả gợi ý giàu ngữ cảnh hơn.

**Bốn là**, xây dựng một **Khung Kiểm thử A/B (A/B Testing Framework)** tích hợp hệ thống Cờ Tính năng (Feature Flags). Lớp Backend cần được thiết kế lại để hỗ trợ tính năng định tuyến lưu lượng (Traffic Routing). Hệ thống sẽ tự động phân phối ngẫu nhiên 50% người dùng trải nghiệm thuật toán Pseudo-Tower và 50% còn lại trải nghiệm thuật toán Lọc Cộng tác truyền thống. Dữ liệu tương tác thực tế từ hai nhóm (Online Metrics) sẽ được gửi về một bảng điều khiển trung tâm để đưa ra các quyết định cập nhật mô hình tự động (CI/CD for ML) dựa trên dữ liệu thống kê (Data-driven decisions) chứ không phải dựa trên cảm tính.

**Năm là**, kết nối dữ liệu thông qua **Sơ đồ Tri thức (Knowledge Graph)**. Thay vì chỉ biểu diễn phim như các vector cô lập, hệ thống tương lai cần triển khai một đồ thị tri thức (Ví dụ: Neo4j) để mô hình hóa mạng lưới quan hệ phức tạp: "Phim X được đạo diễn bởi Đạo diễn Y → Đạo diễn Y thường xuyên hợp tác với Diễn viên Z → Diễn viên Z đóng vai chính trong

Phim W". Việc kết hợp Vector Search và Graph Traversal sẽ cho phép hệ thống tạo ra những chuỗi giải thích (Explainable AI) có logic rõ hơn so với kiến trúc RAG thông thường.

**Cuối cùng**, định hướng mở rộng về mặt hạ tầng mạng là việc phân phối hệ thống thông qua **Điện toán Biên (Edge Computing)**. Thay vì dồn mọi truy vấn về một máy chủ Hugging Face duy nhất, toàn bộ lõi ứng dụng Python có thể được biên dịch lại sang WebAssembly (Wasm) và triển khai trên hạ tầng mạng lưới Cloudflare Workers toàn cầu. Kiến trúc này sẽ đưa động cơ tìm kiếm LanceDB (In-memory) đến gần vị trí địa lý của người dùng hơn, qua đó giảm độ trễ tìm kiếm trên nhiều khu vực triển khai.

## 6.5 Lời kết

Kỷ nguyên Trí tuệ Nhân tạo đang chứng kiến sự thu hẹp khoảng cách giữa nghiên cứu học thuật và khả năng triển khai công nghệ thương mại. Quá trình thực hiện đề tài "Xây dựng hệ thống gợi ý phim ứng dụng Big Data và MLOps" không đơn thuần là việc giải một bài toán về mặt toán học hay tối ưu một vài tham số của mô hình học sâu, mà còn là quá trình rèn luyện tư duy của một Kiến trúc sư Hệ thống (Systems Architect). Đề tài cho thấy giá trị của một mô hình Trí tuệ Nhân tạo không chỉ nằm ở độ phức tạp của số lượng tham số, mà còn nằm ở khả năng phần mềm hóa, đóng gói, luân chuyển và bảo trì mô hình đó trong môi trường đám mây, nơi chi phí, độ trễ và sự cố mạng là các yếu tố luôn phải được tính đến.

Hệ thống Serverless Cloud-native được hiện thực hóa trong đồ án này, dù vẫn có những giới hạn nhất định của một sản phẩm nghiên cứu, đã chứng minh được giá trị của triết lý "Bình dân hóa" (Democratization) Big Data và MLOps. Việc xây dựng các ứng dụng thông minh cấp doanh nghiệp không nhất thiết phải phụ thuộc vào những tập đoàn công nghệ lớn hay các cụm máy chủ GPU đắt đỏ. Bằng cách kết hợp các công cụ mã nguồn mở, khai thác hợp lý các gói dịch vụ miễn phí, và ứng dụng tư duy thiết kế phần mềm sạch (Clean Architecture), nhóm phát triển có thể tạo ra các tác tử AI vận hành liên tục 24/7 với chi phí tiệm cận số 0. Thành công của đồ án là cơ sở để tiếp tục mở rộng nghiên cứu, đồng thời cho thấy tiềm năng ứng dụng của Trí tuệ Nhân tạo trong các hệ thống phần mềm thực tế.



## Tài liệu tham khảo

- [1] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, editors. *Recommender Systems Handbook*. Springer, 2011.
- [2] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182, 2017.
- [4] P. Covington, J. Adams, and E. Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198, 2016.
- [5] F. M. Harper and J. A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.
- [6] GroupLens Research. MovieLens 25M Dataset. <https://grouplens.org/datasets/movielens/25m/>, accessed 2026.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [9] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.
- [10] Sentence-Transformers. Model card: all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, accessed 2026.
- [11] J. Johnson, M. Douze, and H. Jegou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- [12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [13] K. Jarvelin and J. Kekalainen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [14] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, 2015.
- [15] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018.



- [16] LanceDB. LanceDB Documentation. <https://lancedb.github.io/lancedb/>, accessed 2026.
- [17] Hugging Face. Spaces Documentation. <https://huggingface.co/docs/hub/spaces>, accessed 2026.
- [18] Cloudflare. Cloudflare R2 Documentation and Pricing. <https://developers.cloudflare.com/r2/>, accessed 2026.
- [19] Groq. Groq API Documentation. <https://console.groq.com/docs/>, accessed 2026.
- [20] P. T. Tấn. Source code for Big Data MLOps System. <https://github.com/thanhtan2210/Big-Data-MLOps-System>, accessed 2026.